

UNIVERSITATEA BABEȘ-BOLYAI CLUJ-NAPOCA
FACULTATEA DE MATEMATICĂ ȘI INFORMATICĂ
SPECIALIZAREA INFORMATICĂ ÎN LIMBA ROMÂNĂ

LUCRARE DE LICENȚĂ

*Kernel Trap: Sistem Inteligent de Prevenire
a Intruziunilor cu Componente de Honeypot
și Învățare Automată*

Coordonator
Conf. Univ. Dr. *Bușnea Darius Vasile*

Autor
Banciu George-Horațiu

Abstract

Host-based Intrusion Detection Systems (HIDS) and honeypots are two complementary approaches in cyber defense, yet when used in isolation they suffer from significant limitations: the former generate alerts without an automatic response mechanism, while the latter are passive and an attacker who has already compromised a system has no incentive to migrate voluntarily into a trap. This paper presents KernelTrap, a distributed system that integrates both approaches into a single automated pipeline. The system monitors system calls produced by active SSH sessions via eBPF technology, evaluates each event using an Isolation Forest model trained on the BETH dataset, and when a sliding window signals a critical level of anomalies, transparently redirects the attacker’s session to a Docker honeypot container. Experimental results on real-world attack scenarios confirm the feasibility of the approach, with CPU overhead below 2.5

Cuprins

1	Introducere	6
2	Aspecte teoretice	10
2.1	Sisteme de detectare și prevenire a intruziunilor	10
2.2	Apeluri de sistem și rolul lor în securitate	11
2.3	Extended Berkeley Packet Filter (eBPF)	12
2.4	Tracee: instrumentul de colectare a evenimentelor	13
2.5	Honeypot-uri și apărarea activă	14
2.6	Algoritmul Isolation Forest	15
2.7	Setul de date BETH	16
2.8	Redis Streams	17
2.9	Containerizarea Docker și rolul în mecanismul honeypot	17
3	Stadiu actual	19
3.1	Sisteme HIDS bazate pe eBPF	19
3.2	Învățare automată nesupervizată pentru detectarea intruziunilor	20
3.3	Praguri adaptive și agregare temporală	22
3.4	Honeypot-uri SSH cu interacțiune ridicată	22
3.5	Apărare activă și pivotare transparentă	23
3.6	Sisteme integrate similare KernelTrap	24
3.7	Sinteză și poziționarea KernelTrap	24
4	Abordarea propusă	26
4.1	Vedere de ansamblu	27
4.2	Colectarea și preprocesarea evenimentelor la nivel de agent	28
4.3	Infrastructura de comunicație prin Redis Streams	29
4.4	Modelul de detecție a anomaliilor	29
4.4.1	Caracteristici și antrenare	30
4.4.2	Praguri și calibrare în producție	30
4.5	Agregarea prin fereastră glisantă	31
4.6	Mecanismul de pivotare	32

4.6.1	Detectie și comandă	32
4.6.2	Execuția pivotării	32
4.7	Honeypot-ul	33
4.7.1	Oglindire dinamică a identității	33
4.7.2	Fișiere canary	33
4.7.3	Înregistrarea sesiunilor TTY	34
4.7.4	Monitorizarea în container și feedback loop	34
4.8	Panoul de control în timp real	35
5	Aplicația	36
5.1	Cerințe	36
5.2	Analiză	37
5.3	Proiectare	37
5.4	Implementare	38
5.4.1	Tehnologii și organizarea proiectului	38
5.4.2	Componente	40
5.5	Testare	42
6	Evaluare	45
6.1	Asumări de securitate	45
6.2	Mediul de testare și metodologie	46
6.3	Performanță	46
6.4	Scenarii de atac și răspunsul sistemului	46
6.4.1	Scenariul 1: recon automat cu LinPEAS	47
6.4.2	Scenariul 2: stabilirea unui canal C2 cu Sliver	48
6.4.3	Scenariul 3: acces la fișiere canary post-pivotare	49
6.5	Limitări	49
6.5.1	Domeniul de aplicabilitate optim	50
6.6	Sinteză evaluare	51
7	Concluzii și direcții viitoare	52

Listă de figuri

2.1	Fluxul de colectare prin eBPF	13
2.2	Intuiția algoritmului Isolation Forest	15
4.1	Fluxul de date în KernelTrap	28
5.1	Arhitectura de componente	38
5.2	Diagrama de secvență a procesului de detecție și pivotare	39
5.3	Distribuția scorurilor Isolation Forest	40
6.1	Overhead-ul CPU al agentului de colectare	47

Listă de tabele

3.1	Comparație între soluțiile existente și KernelTrap pe cele patru dimensiuni cheie	25
5.1	Cerințe funcționale	36
5.2	Cerințe nefuncționale	37
5.3	Stiva tehnologică pe componente	39
6.1	Asumările de securitate ale sistemului	45
6.2	Configurația mediului de testare	46
6.3	Metrice de performanță	47
6.4	Scenariul 1: indicatori observați la rularea LinPEAS	47
6.5	Scenariul 2: indicatori observați la setup Sliver	48
6.6	Scenariul 3: indicatori observați la accesul canary	49
6.7	Profile de atacator și acoperirea KernelTrap	51

Capitolul 1

Introducere

Creșterea numărului de amenințări cibernetice și sofisticarea atacurilor direcționate impun soluții de securitate capabile să depășească limitele modelelor tradiționale de detecție. Sistemele de detectare a intruziunilor bazate pe gazdă (*Host-based Intrusion Detection Systems*, HIDS) oferă un strat de apărare cu granularitate ridicată, monitorizând evenimentele produse direct pe sistemul de operare al mașinii țintă [35]. Printre sursele de telemetrie disponibile, apelurile de sistem (*syscalls*) reprezintă cea mai fidelă oglindă a comportamentului proceselor, deoarece orice acțiune relevantă pentru securitate, crearea de procese, accesul la fișiere, comunicația prin rețea, trece obligatoriu prin această interfață. Metodele de învățare automată aplicate pe secvențe de *syscalls* depășesc abordările clasice bazate pe semnături, deoarece pot detecta și atacuri necunoscute anterior (*zero-day*) [42]; compromisul acceptat este o rată mai mare de fals pozitive, care trebuie gestionată prin mecanisme suplimentare de agregare.

Această compensare între acoperire și precizie este caracteristică detecției comportamentale: un model care învață doar tiparele normale de activitate va semnaliza drept suspectă orice abatere, inclusiv operații legitime dar rare, cum ar fi o sarcină de mentenanță sau o actualizare neobișnuită de sistem. Din acest motiv, un scor de anomalie izolat nu poate constitui, singur, temei pentru o acțiune automată; el trebuie corelat în timp cu alte semnale, astfel încât deciziile sistemului să se bazeze pe un tipar persistent de comportament anormal, nu pe un eveniment accidental. **KernelTrap** adoptă tocmai această strategie, agregând scorurile pe o fereastră temporală înainte de a lua orice decizie de răspuns.

O categorie complementară de apărare o constituie honeypot-urile, sisteme-capcană proiectate să atragă atacatorii într-un mediu controlat, izolat de resursele reale. Abordările tradiționale de honeypot sunt însă pasive: atacatorul trebuie să ajungă la capcană din proprie inițiativă, ceea ce nu se întâmplă dacă a compromis deja un sistem legitim. Lucrarea de față pornește de la această limitare și propune **KernelTrap**: un sistem activ de prevenire a intruziunilor care, în momentul în care detectează un comportament anormal, redirectionează transparent sesiunea SSH a atacatorului către un

container honeypot dedicat, fără a-l avertiza.

Limitarea pasivității nu ține de calitatea capcanei, ci de modelul de interacțiune: oricât de convingător ar fi un honeypot, el rămâne inert atâta timp cât atacatorul nu are niciun motiv să-l caute. În scenariul vizat de această lucrare, un cont SSH legitim a fost deja compromis, iar atacatorul operează direct pe sistemul real; el nu va părăsi voluntar o gazdă funcțională pentru a explora una necunoscută. Soluția propusă inversează această dinamică: în locul unui honeypot care așteaptă să fie descoperit, sistemul detectează comportamentul ostil pe gazda reală și mută el însuși sesiunea atacatorului în mediul-capcană, transformând honeypot-ul dintr-o țintă opțională într-o destinație impusă, fără ca atacatorul să sesizeze tranziția.

Bazat pe o analiză a literaturii de specialitate și a tehnologiilor disponibile, sistemul KernelTrap integrează trei niveluri funcționale: colectarea evenimentelor de syscall prin intermediul tehnologiei *extended Berkeley Packet Filter* (eBPF) cu instrumentul Tracee, analiza centralizată bazată pe algoritmul Isolation Forest antrenat pe setul de date BETH, și un mecanism de răspuns activ care combină semnale POSIX, containere Docker și reguli `iptables`.

Spre deosebire de soluțiile existente examinate în capitolul 3, care acoperă doar o parte din spectrul de funcționalități (fie monitorizare la nivel de kernel fără componentă de capcanare, fie învățare automată împreună cu honeypot dar fără răspuns activ, fie pivotare reactivă fără strat de detecție comportamentală), KernelTrap integrează simultan toate cele patru dimensiuni: telemetrie de kernel prin eBPF, atribuirea scorurilor prin învățare automată nesupervizată, honeypot cu interacțiune ridicată și pivotare transparentă inițiată automat de sistem. Această integrare nu reprezintă o sumă mecanică de componente, ci permite o buclă completă de tip detecție, decizie și izolare, în care atacatorul autentificat este captat fără a sesiza tranziția. Evaluarea experimentală a confirmat fezabilitatea abordării pe trei scenarii reale de atac, pivotarea fiind declanșată corect în fiecare caz, cu un overhead pe CPU al gazdei monitorizate de aproximativ 1,8% sub încărcare, sub ținta de 2,5% stabilită ca cerință nefuncțională.

Protecția oferită de KernelTrap este definită în raport cu un scenariu de atac precis și cu două asumări de securitate explicite, detaliate în capitolul 6. În primul rând, se presupune că un cont SSH neprivilegiat a fost compromis pe o gazdă altfel sănătoasă, în care contul root rămâne sub control; un atacator care a obținut deja drepturi de administrator ar putea dezactiva agentul însuși. În al doilea rând, se presupune că atacatorul nu are acces fizic la mașină, astfel încât vectorii prin consolă locală sau hardware rămân în afara domeniului de aplicabilitate. Aceste asumări nu reprezintă o slăbiciune, ci o delimitare deliberată a clasei de atacatori vizate, anume atacatorul autentificat de nivel mediu.

Contribuțiile principale ale lucrării pot fi rezumate astfel:

1. un agent de colectare bazat pe eBPF și Tracee, ușor și fără stare, care filtrează

telemetria la nivelul sesiunilor SSH active și o normalizează într-un format consistent între gazde;

2. un model Isolation Forest antrenat pe setul de date BETH, în care setul clasic de șapte câmpuri numerice este extins la 22 de caracteristici, obținând o valoare AUROC de 0,969 pe partiția de test;
3. un mecanism de agregare temporală prin fereastră glisantă cu prag dublu, de cantitate și de rată, care reduce rata de fals pozitive față de un prag pur cantitativ;
4. un mecanism de pivotare transparentă inițiat automat de sistem, care redirecționează sesiunea SSH compromisă către un container honeypot fără a întrerupe conexiunea și fără a alerta atacatorul;
5. un panou de control în timp real care expune starea agenților, fluxul de evenimente scoruite și istoricul pivotărilor, completând automatizarea cu posibilitatea de intervenție manuală.

Lucrarea este organizată în șapte capitole, după cum urmează:

- Capitolul 1 prezintă contextul problemei, motivația și contribuțiile principale ale lucrării.
- Capitolul 2 introduce conceptele teoretice necesare înțelegerii sistemului: apelurile de sistem și rolul lor în securitate, tehnologia eBPF și Tracee, algoritmul Isolation Forest, Redis Streams și containerizarea Docker.
- Capitolul 3 analizează lucrările relevante din domeniu, acoperind HIDS bazate pe syscall-uri [35], metode de învățare automată aplicate la detectarea intruziunilor [42], precum și soluțiile existente de tip honeypot și limitările acestora.
- Capitolul 4 descrie arhitectura detaliată a sistemului KernelTrap: deciziile de proiectare, protocoalele de comunicație inter-componente și fluxul de date de la colectare până la răspuns.
- Capitolul 5 prezintă implementarea concretă a fiecărei componente: agentul de colectare, serverul central de analiză și mecanismul de pivotare către honeypot.
- Capitolul 6 prezintă evaluarea experimentală a sistemului: metrice de performanță, scenarii reale de atac cu unelte publice și limitările asumate ale domeniului de aplicabilitate.
- Capitolul 7 sintetizează contribuțiile lucrării și propune direcții de extindere ulterioară a sistemului.

În procesul de elaborare a acestei lucrări, autorul a utilizat modele lingvistice generative, în special ChatGPT și Claude Code, pentru asistarea completării unor fragmente de cod, generarea unor versiuni inițiale ale codului și, ocazional, reformularea unor enunțuri pentru sporirea clarității. După utilizarea acestor instrumente, conținutul a fost revizuit și editat de autor, care își asumă întreaga responsabilitate pentru forma finală și pentru conținutul tezei.

Capitolul 2

Aspecte teoretice

Capitolul de față prezintă fundamentele conceptuale și tehnologice pe care se construiesc sistemele moderne de detectare și prevenire a intruziunilor bazate pe monitorizarea la nivel de kernel. Sunt tratate paradigmele IDS/IPS, mecanismele de interceptare a apelurilor de sistem prin eBPF, instrumentele de colectare a evenimentelor, tehnicile de detecție a anomaliilor prin învățare automată nesupervizată și principiile apărării active prin honeypot-uri.

2.1 Sisteme de detectare și prevenire a intruziunilor

Un *sistem de detectare a intruziunilor* (*Intrusion Detection System*, IDS) este o componentă de securitate ce monitorizează în timp real activitatea unui sistem sau rețea în scopul identificării comportamentelor suspecte sau a activităților neautorizate [35]. Atunci când un IDS este dotat și cu capacități de răspuns activ (blocarea conexiunilor, terminarea proceselor sau redirecționarea traficului), acesta devine un *sistem de prevenire a intruziunilor* (*Intrusion Prevention System*, IPS).

Din perspectiva punctului de colectare a datelor, sistemele IDS se clasifică în două mari categorii:

- NIDS (*Network-based IDS*): monitorizează traficul de rețea la nivel de pachete, identificând tipare de atac (scanări de porturi, flood-uri, exploit-uri cunoscute) fără acces la starea internă a gazdelor individuale.
- HIDS (*Host-based IDS*): rulează direct pe gazda monitorizată și analizează evenimente produse la nivelul sistemului de operare (apeluri de sistem, jurnale de aplicație, accesuri la fișiere și modificări de configurație), oferind vizibilitate asupra comportamentului proceselor individuale [35].

Cele două abordări sunt complementare: NIDS detectează atacuri la nivel de transport, dar nu poate inspecta comunicațiile criptate și nu are acces la comportamentul

intern al proceselor; HIDS are vizibilitate deplină asupra stării gazdei, dar nu observă activitatea laterală dintre alte mașini din rețea.

Literatura de specialitate distinge două paradigme principale de detecție [35, 42]:

Detecția bazată pe semnături (*signature-based detection*) compară activitatea observată cu o bază de cunoștințe de tipare de atac cunoscute. Rata de fals pozitive este scăzută iar detecția este rapidă, dar metoda este prin definiție oarbă la amenințările noi (*zero-day*) care nu figurează în baza de semnături. Soluții comerciale precum Snort sau Suricata se bazează preponderent pe această abordare.

Detecția bazată pe anomalii (*anomaly-based detection*) construiește un model al comportamentului normal și semnalează orice deviere semnificativă de la acesta. Această paradigmă poate detecta, în principiu, atacuri necunoscute anterior, deoarece nu depinde de semnăturile lor specifice. Compromisul acceptat constă într-o rată mai ridicată de fals pozitive, generată de variabilitatea naturală a comportamentului benign. Sistemele moderne combină adesea ambele abordări pentru a valorifica avantajele fiecăreia [42].

2.2 Apeluri de sistem și rolul lor în securitate

Sistemul de operare Linux separă execuția în *user space* (unde rulează aplicațiile neprivilegiate) și *kernel space*, unde se execută codul privilegiat al nucleului. Această separare este impusă hardware prin inelele de protecție ale procesorului (Ring 0 pentru kernel, Ring 3 pentru aplicații) și constituie fundația modelului de securitate al sistemului de operare [15].

Comunicarea dintre cele două domenii se realizează exclusiv prin *apeluri de sistem* (*syscalls*): puncte de intrare controlate prin care un program solicită nucleului operații privilegiate. Nucleul Linux expune în prezent peste 300 de apeluri de sistem, grupate funcțional în categorii: gestionarea proceselor (*fork*, *execve*, *exit*), accesul la fișiere (*open*, *read*, *write*, *unlink*), comunicația prin rețea (*socket*, *connect*, *bind*), gestionarea utilizatorilor și privilegiilor (*setuid*, *setgid*, *capset*) și controlul proceselor (*kill*, *ptrace*) [29].

Orice acțiune relevantă din perspectiva securității trece obligatoriu prin interfața *syscall*. Un atacator care rulează un exploit, escaladează privilegii sau exfiltrează date nu poate ocoli această interfață, indiferent de tehnica folosită. Această proprietate face din secvențele de *syscall*-uri o sursă ideală de telemetrie pentru sistemele HIDS.

Spre deosebire de logurile aplicative (care pot fi falsificate sau suprimate de un atacator cu privilegii ridicate), evenimentele de la nivelul *syscall*-urilor sunt inspecționate de nucleu înainte ca aplicația să aibă posibilitatea de a interveni [11]. Astfel, chiar dacă un atacator compromite complet o aplicație, comportamentul acesteia rămâne vizibil prin instrumentarea nucleului.

Din perspectiva unui HIDS, secvențele de syscall-uri permit identificarea unor tipare comportamentale caracteristice atacurilor: un `execve` urmat imediat de `setuid` și `capset` sugerează o tentativă de escaladare de privilegii; o explozie de apeluri `connect` către adrese IP externe indică o potențială activitate de tip Command and Control; o secvență `open/read` pe fișierele `/etc/shadow` sau `/etc/passwd` semnalează recunoaștere locală a sistemului [35].

2.3 Extended Berkeley Packet Filter (eBPF)

Berkeley Packet Filter (BPF), introdus în 1992 de McCanne și Jacobson ca mecanism de filtrare a pachetelor de rețea la nivel de kernel [37], a fost extins radical începând cu nucleul Linux 3.18 (2014) sub forma *extended BPF* (eBPF) [23]. Dacă BPF-ul clasic opera exclusiv pe pachete de rețea, eBPF permite atașarea de programe la orice punct de instrumentare din nucleu: syscall-uri, kprobes, uprobes, tracepoints și evenimente LSM (*Linux Security Modules*).

Arhitectural, eBPF funcționează ca o mașină virtuală ușoară integrată în nucleu, cu un set de instrucțiuni propriu (64 de biți, 11 regiștri generali) și un model de execuție bazat pe evenimente. Programele sunt scrise în C restricționat, compilate în bytecode și supuse unui *verificator static* care garantează absența buclelor infinite, a acceselor invalide la memorie și a scurgerilor de resurse [23]. Verificatorul analizează toate căile posibile de execuție ale programului înainte de încărcare, respingând programele care nu pot fi demonstrate sigure. Bytecode-ul validat este ulterior compilat JIT (*Just-in-Time*) în instrucțiuni native ale procesorului.

Starea este partajată între programe și user space prin *eBPF Maps*, structuri cheie-valoare rezidente în kernel space, accesibile din ambele domenii. Apelurile din programele eBPF către funcțiile nucleului se fac exclusiv prin *helper functions* cu interfață stabilă, garantând compatibilitatea între versiuni diferite de kernel. Figura 2.1 sintetizează acest flux, de la apelul de sistem produs în spațiul utilizator până la citirea evenimentului filtrat, într-un map, de către un agent din spațiul utilizator precum Tracee.

Înainte de eBPF, instrumentarea nucleului la runtime necesita una din două abordări, ambele problematice în producție: modificarea codului sursă al nucleului (imposibilă fără acces la sursă și fără repornire) sau încărcarea de module de nucleu (*Loadable Kernel Modules*, LKM). Modulele de nucleu rulează în Ring 0, fără niciun mecanism de verificare prealabilă, iar un bug poate produce panică de kernel sau compromiterea întregului sistem.

eBPF elimină aceste limitări: programele se încarcă la runtime, fără repornire, și sunt garantate sigure de verificator înainte de execuție. Față de mecanisme alternative precum `auditd` sau `ptrace`, eBPF prezintă cost suplimentar semnificativ mai redus,

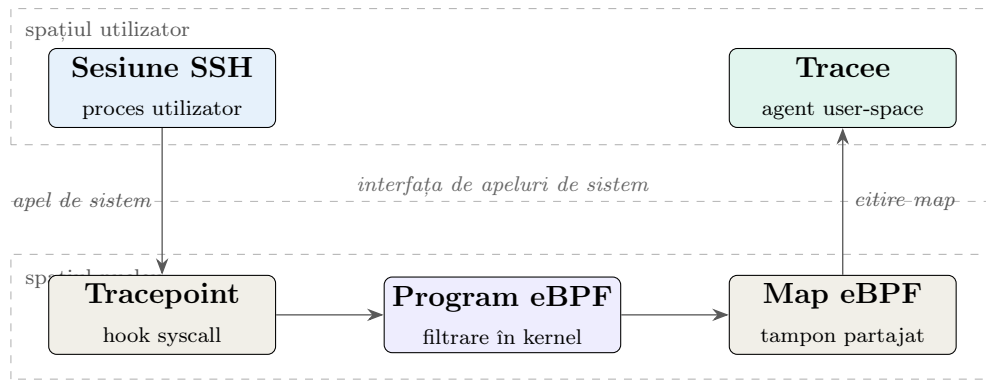


Figura 2.1: Fluxul de colectare prin eBPF. Un apel de sistem produs de o sesiune SSH traversează interfața de apeluri de sistem; în nucleu, un program eBPF atașat la un tracepoint filtrează evenimentul și îl scrie într-un map, de unde agentul Tracee îl citește în spațiul utilizator.

deoarece filtrarea și agregarea evenimentelor se realizează direct în kernel space, reducând la minimum cantitatea de date transferate către user space [23].

2.4 Tracee: instrumentul de colectare a evenimentelor

Tracee este un instrument open-source de securitate runtime pentru Linux, dezvoltat și menținut de Aqua Security [12]. Folosește eBPF pentru a intercepta evenimentele de sistem la runtime și a le expune ca fluxuri de date structurate în format JSON, acoperind un spectru larg de activitate: crearea și terminarea de procese, accesul la fișiere, conexiunile de rețea, modificările de permisiuni și evenimentele LSM.

Tracee este compus din două componente principale: **Tracee-eBPF**, care compilează și încarcă programele eBPF în nucleu și expune fluxul brut de evenimente, și **Tracee-Rules**, un motor de detectare bazat pe reguli scrise în Go sau Rego (limbajul de politici al Open Policy Agent). O caracteristică notabilă este preferința pentru evenimentele LSM față de syscall-urile brute (de exemplu, `security_file_open` în locul `openat`), care oferă informații mai complete despre contextul de securitate și sunt mai puțin susceptibile la atacuri de tip TOCTOU (*Time-of-Check to Time-of-Use*) [12].

Portabilitatea pe kernel-uri diferite este asigurată prin suportul *Compile Once – Run Everywhere* (CO:RE), bazat pe *BPF Type Format* (BTF): pe nucleele cu BTF activat, nu sunt necesare nici recompilarea probe-urilor, nici prezența header-elor de nucleu pe sistemul țintă. Aceasta face din Tracee o soluție practică pentru deployment pe infrastructuri eterogene.

2.5 Honeypot-uri și apărarea activă

Un *honeypot* este un sistem de securitate proiectat deliberat să fie compromis, cu scopul de a atrage atacatorii, de a le studia tehnicile și de a le deturna resursele și atenția de la sistemele reale [46]. Spre deosebire de sistemele de producție, un honeypot nu conține date sau servicii reale, iar orice interacțiune cu el este, prin definiție, suspectă.

Taxonomia standard clasifică honeypot-urile după două criterii principale:

Nivelul de interacțiune distinge între:

- *Honeypot-uri cu interacțiune redusă (low-interaction)*: emulează servicii și sisteme de operare fără a rula un sistem real. Sunt ușor de implementat și cu risc minim de compromitere, dar oferă informații limitate despre tehnicile atacatorului. Exemple tipice: Honeyd, OpenCanary.
- *Honeypot-uri cu interacțiune ridicată (high-interaction)*: expun sisteme reale (sau mașini virtuale complete) cu servicii funcționale. Colectează informații detaliate despre comportamentul atacatorului, inclusiv tool-uri folosite, comenzi executate și ținte vizate ulterior, dar prezintă risc mai ridicat dacă atacatorul reușește să evadeze din mediul controlat [46].

Scopul de utilizare distinge între:

- *Honeypot-uri de cercetare*: implementate de organizații de securitate pentru studiarea tehnicilor de atac emergente, colectarea de mostre de malware și înțelegerea comportamentului atacatorilor.
- *Honeypot-uri de producție*: integrate în rețelele organizațiilor pentru a detecta activitate neautorizată internă și a alerta echipele de securitate.

Honeypot-urile tradiționale funcționează pasiv: atacatorul trebuie să ajungă la capcană din proprie inițiativă, alegând să interacționeze cu un serviciu-momeală. Această abordare prezintă o limitare fundamentală în scenariile în care atacatorul a compromis deja un sistem legitim, deoarece el nu are niciun motiv să migreze voluntar către un honeypot.

O abordare mai avansată, denumită *apărare activă (active deception)*, inversează această logică prin redirectionarea transparentă a unui atacator detectat de pe sistemul compromis către un honeypot, fără ca acesta să conștientizeze tranziția. Mecanismul se bazează tipic pe reguli NAT (`iptables/nftables`), care interceptează traficul la nivel de rețea și îl deviază înainte ca pachetele să ajungă la destinația originală [46]. Avantajul față de abordarea pasivă constă în faptul că atacatorul nu trebuie să aleagă honeypot-ul din proprie inițiativă, ci este direcționat acolo în momentul în care comportamentul său declanșează un semnal de alertă, indiferent că a compromis deja un serviciu real.

Un concept complementar honeypot-urilor îl reprezintă *honeypot-ul*: un artefact digital fals (fișier, cheie de acces, înregistrare în bază de date) plasat intenționat într-un sistem real sau într-un honeypot, proiectat să nu fie niciodată accesat de utilizatori legitimi. Orice interacțiune cu un honeypot constituie, prin definiție, un indicator incontestabil de compromitere [46]. Spre deosebire de detecția bazată pe anomalii statistice, care produce un scor continuu cu risc de fals pozitive, accesul la un honeypot este un semnal binar și precis: niciun utilizator legitim nu are motiv să deschidă un fișier de credențiale pe care el însuși nu l-a creat. Honeypot-uri pot lua diverse forme: fișiere cu credențiale fictive, chei SSH false, token-uri de API nevalide sau înregistrări-capcană în baze de date. Sunt utilizate atât în medii de producție pentru detectarea accesului neautorizat, cât și în honeypot-uri pentru a monitoriza comportamentul post-compromitere al atacatorilor.

2.6 Algoritmul Isolation Forest

Isolation Forest (*iForest*), propus de Liu, Ting și Zhou în 2008 [33], adoptă o paradigmă diferită față de metodele clasice de detecție a anomaliilor (LOF, k-NN, One-Class SVM): în loc să modeleze distribuția normală și să identifice instanțele care se abat de la ea, izolează explicit anomaliile prin partiționări recursive aleatorii.

Intuiția algoritmului se bazează pe două proprietăți ale anomaliilor: acestea sunt *rare* (puține la număr față de instanțele normale) și *diferite* (ocupă regiuni izolate ale spațiului de caracteristici). Ca urmare, ele pot fi separate de restul datelor printr-un număr semnificativ mai mic de partiționări aleatorii decât instanțele normale, care sunt dense și necesită multe partiționări pentru a fi izolate [34] (Figura 2.2).

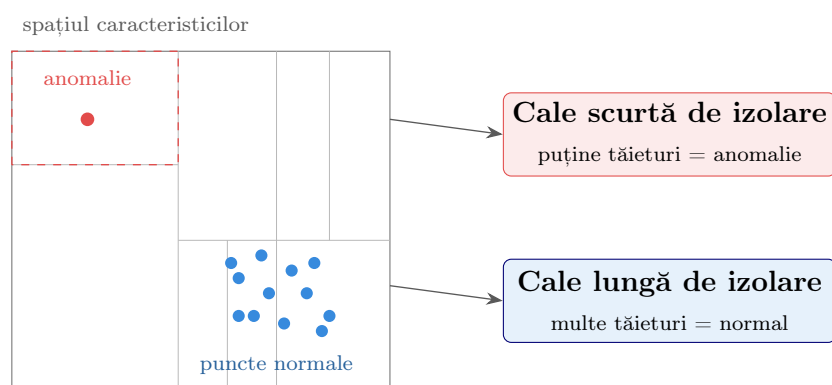


Figura 2.2: Intuiția algoritmului Isolation Forest. O anomalie, situată într-o regiune rară a spațiului de caracteristici, este izolată din puține tăieturi aleatorii (cale scurtă în arbore), în timp ce un punct dintr-un cluster dens necesită multe tăieturi (cale lungă) pentru a fi separat.

Algoritmul construiește o pădure de *isolation trees* (iTrees). Fiecare iTree este un arbore binar construit prin selecția aleatorie repetată a unui atribut q și a unui

prag de partiționare p ales uniform în intervalul de valori al atributului, până când fiecare instanță este izolată într-un nod frunză. O pădure constă din t astfel de arbori, fiecare antrenat pe un subsample de dimensiune ψ ; valorile implicite sunt $t = 100$, $\psi = 256$ [33].

Lungimea căii $h(x)$ printr-un iTree reprezintă numărul de muchii parcurse de la rădăcină până la nodul de izolare al instanței x . Anomaliile, mai ușor de izolat, prezintă căi mai scurte; instanțele normale, mai greu de separat, prezintă căi mai lungi. Scorul de anomalie normalizat este definit ca:

$$s(x, \psi) = 2 - \frac{E[h(x)]}{c(\psi)}$$

unde $E[h(x)]$ este media lungimilor de cale pe toți t arborii, iar $c(\psi)$ este lungimea medie de cale așteptată pentru un arbore binar de căutare construit pe ψ instanțe [34]. Un scor $s \approx 1$ indică anomalie certă, $s \approx 0.5$ instanță nedistinctivă, iar $s \approx 0$ comportament cu certitudine normal.

Față de metodele bazate pe distanță sau densitate, iForest are complexitate liniară $O(n)$, memorie redusă prin subsampling fix și nu necesită date etichetate cu atacuri, proprietăți esențiale într-un context de securitate unde datele de atac sunt rare sau inexistente la momentul antrenării [33, 34].

2.7 Setul de date BETH

Setul de date BETH (*BPF-Extended Tracking Honeypot*), prezentat de Highnam et al. la ICML Workshop on Uncertainty and Robustness in Deep Learning (2021) [25], este unul dintre puținele seturi de date publice de evenimente de syscall colectate dintr-un scenariu real de honeypot în mediu cloud. Conține peste 8 milioane de evenimente capturate de pe 23 de gazde honeypot expuse pe internet timp de mai multe zile.

Fiecare eveniment este reprezentat prin 14 caracteristici brute: `processId`, `parentProcessId`, `userId`, `mountNamespace`, `eventId`, `argsNum`, `returnValue` și altele, și este etichetat binar: benign (`evil=0`) sau malițios (`evil=1`). Setul este partiționat 60/20/20 (antrenament/validare/test); important, datele de atac apar *exclusiv* în partiția de test, reflectând paradigma realistă în care un model este antrenat pe comportament benign și evaluat ulterior pe atacuri. Atacul reprezentat constă în instanțierea unui nod de botnet: exploatare inițială, escaladare de privilegii și activitate de Command and Control (C&C).

Față de alte seturi de date sintetice sau de laborator, BETH prezintă avantajul că reflectă comportamentul real al unui atacator pe servere Linux expuse la internet, colectat prin aceeași tehnologie eBPF utilizată de instrumentele moderne de monitori-

zare. Isolation Forest atinge AUROC de 0.850 pe acest set, cel mai bun rezultat dintre modelele clasice nesupervizate testate în literatura de specialitate [25].

2.8 Redis Streams

Redis Streams, introdusă în Redis 5.0 (2018), este o structură de date de tip jurnal *append-only* cu acces aleator în timp $O(1)$, destinată scenariilor de mesagerie și procesare de evenimente în timp real [40]. Fiecare intrare (*entry*) dintr-un stream este identificată printr-un ID monoton crescător (compus din timestamp în milisecunde și un număr de secvență) și conține un set arbitrar de perechi câmp-valoare.

Spre deosebire de Redis Pub/Sub, care pierde mesajele dacă niciun subscriber nu este activ în momentul publicării, Streams persistă intrările pe disc și le face disponibile pentru citire ulterioară. Mecanismul de *consumer groups* permite mai multor consumatori să citească din același stream în paralel, fiecare primind un subset disjunct de mesaje; un mesaj rămâne în starea *pending* până la confirmarea explicită prin comanda **XACK**, garantând că niciun eveniment nu se pierde în caz de cădere a consumatorului.

Aceste caracteristici fac din Redis Streams o soluție potrivită pentru arhitecturi de tip pipeline în care un producător de evenimente (agentul de monitorizare) trebuie decuplat de consumatorul de analiză, cu garanții de livrare și fără introducerea complexității operaționale a unui broker distribuit dedicat.

2.9 Containerizarea Docker și rolul în mecanismul honeypot

Docker permite ambalarea unei aplicații împreună cu toate dependențele sale într-o unitate portabilă și reproductibilă numită *container* [38]. Spre deosebire de mașinile virtuale, care virtualizează hardware-ul complet și rulează un nucleu propriu, containerele partajează nucleul sistemului gazdă și se izolează exclusiv prin mecanisme software ale nucleului Linux:

- *Namespaces*: creează viziuni izolate ale resurselor sistemului pentru fiecare container: spații separate de PID (proceseale unui container nu pot vedea procesele altui container), rețea (interfețe și tabele de rutare proprii), sistem de fișiere (*mount namespace*), utilizatori (*user namespace*) și hostname.
- *Control groups (cgroups)*: limitează cantitatea de resurse (CPU, memorie, I/O pe disc, lățime de bandă) pe care un container o poate consuma, prevenind consumul abuziv al resurselor gazdei.

Izolarea prin containere se dovedește utilă și în contextul apărării active: un container honeypot cu interacțiune ridicată poate expune un shell real și unelte uzuale, simulând un sistem Linux funcțional, fără ca activitatea atacatorului să afecteze infrastructura de producție. Tranzitional, regulile NAT (`iptables`, tabela `nat`, lanțul `PREROUTING`) pot intercepta traficul destinat unui serviciu legitim și îl pot devia transparent către portul corespunzător al containerului honeypot, înainte ca pachetele să ajungă la destinația originală. Din perspectiva atacatorului, sesiunea continuă fără întrerupere vizibilă, dar el interacționează cu un mediu controlat și monitorizat, izolat complet de sistemele de producție [29, 46].

Capitolul 3

Stadiu actual

Capitolul de față trece în revistă stadiul actual al cercetării în domeniile care converg în arhitectura sistemului propus, evaluează soluțiile existente și identifică golul pe care lucrarea de față urmărește să îl acopere. KernelTrap se situează la intersecția a patru direcții de cercetare active în literatura de specialitate din ultimii ani: monitorizarea comportamentală la nivel de kernel prin tehnologia eBPF, detectarea anomaliilor pe baza apelurilor de sistem prin metode de învățare automată nesupervizată, honeypot-urile cu interacțiune ridicată pentru analiza atacurilor reale și mecanismele de apărare activă bazate pe redirectionare transparentă a sesiunilor compromise. Pentru fiecare direcție sunt prezentate atât lucrările fundamentale, cât și contribuțiile recente (2022–2025), ale căror rezultate fundamentează deciziile de proiectare expuse în capitolul următor.

3.1 Sisteme HIDS bazate pe eBPF

Tranziția dinspre modulele de nucleu tradiționale (LKM) către programe eBPF reprezintă cea mai semnificativă schimbare în domeniul monitorizării de securitate la nivel de gazdă din ultimul deceniu. Soldani et al. [45] oferă cea mai cuprinzătoare analiză recentă a tehnologiei eBPF pentru observabilitate cloud-native și securitate runtime, demonstrând că suprafața redusă de atac, garanțiile statice oferite de verifcator și costul redus de instrumentare fac din eBPF alegerea naturală pentru sisteme HIDS moderne. Hao et al. [24] descriu în detaliu implementarea runtime-ului eBPF în nucleul Linux 6.7, confirmând maturitatea ecosistemului ca platformă stabilă pentru produse de securitate.

În spațiul soluțiilor open-source, patru proiecte domină piața: **Falco** (CNCF, originar de la Sysdig), care folosește eBPF pentru colectare și un motor de reguli declarative pentru detecție; **Tetragon** (Cilium/Isovalent), care diferă prin capacitatea de aplicare sincronă a politicilor direct în kernel space; **KubeArmor** (Accuknox), care exploatează BPF-LSM pentru aplicarea de politici de securitate; și **Tracee** (Aqua Security),

care a fost adoptat și de KernelTrap pentru capacitățile sale de tracing fără configurare prealabilă pe nuclee diferite (CO:RE/BTF). Syairozi și Arizal [47] efectuează cea mai recentă comparație empirică a acestor sisteme pe un set de atacuri OWASP Kubernetes Top 10, măsurând timpul mediu până la detecție (MTTD), rata de detecție, rata de fals pozitive și consumul de resurse. Concluzia studiului este că Tracee oferă cea mai bună acoperire a evenimentelor LSM și cea mai mică amprentă pe CPU dintre soluțiile testate, susținând alegerea făcută în cadrul prezentei lucrări.

Două lucrări recente sunt deosebit de relevante pentru arhitectura KernelTrap. eBPF-PATROL [6] propune un agent runtime ușor pentru interceptarea apelurilor de sistem și aplicarea de politici, raportând un overhead sub 2,5% un punct de referință apropiat de obiectivele de performanță ale agentului din KernelTrap. CryptoGuard [3], publicat la DSN 2024, combină monitorizarea cu fereastră glisantă a syscall-urilor cu o rețea neuronală evaluată direct în kernel, atingând un overhead remarcabil de 0,06% pe CPU. Aceste lucrări demonstrează fezabilitatea sistemelor HIDS de tip *always-on* în medii de producție, însă niciuna nu integrează un mecanism de răspuns activ comparabil cu pivotarea propusă de KernelTrap.

O contribuție notabilă este lucrarea fundamentală a lui Fournier et al. [22] de la Datadog, care discută în detaliu compromisurile dintre kprobe-uri, tracepoint-uri și hook-urile LSM, precum și problemele de tip TOCTOU (*Time-of-Check to Time-of-Use*) aspecte direct relevante pentru proiectarea agentului KernelTrap.

3.2 Învățare automată nesupervizată pentru detectarea intruziunilor

Limitările abordărilor supervizate aplicate la detectarea intruziunilor sunt bine documentate în literatură. Zoppi și Ceccarelli [51, 52] argumentează că modelele bazate pe semnături și clasificatorii binari supervizați eșuează sistematic la detectarea atacurilor zero-day, întrucât distribuția datelor de atac din timpul antrenării nu reflectă varietatea atacurilor întâlnite în producție. Studiul lor de pe TON_IoT și CICIDS demonstrează că un detector nesupervizat (în particular Isolation Forest) păstrează o sensibilitate reală pe atacuri necunoscute, în timp ce clasificatorii supervizați colapsează la performanțe sub nivelul aleatoriu pe această clasă de scenarii.

Această observație a generat o linie consistentă de cercetare bazată pe Isolation Forest [33] și variante ale acestuia. Studiul comparativ publicat în 2025 în jurnalul *Computers* [7] testează direct OCSVM, Autoencoder, VAE și Isolation Forest pe seturi de date industriale, raportând că IF oferă cel mai bun compromis între acuratețe, complexitate computațională și interpretabilitate. Anand et al. [10] confirmă rezultatul pe TON_IoT, demonstrând superioritatea IF față de OCSVM pe date de înaltă

dimensionalitate cu dezechilibre puternice între clase.

În domeniul specific al apelurilor de sistem, cercetarea modernă se grupează în jurul setului de date BETH [25], introdus de Highnam et al. în 2021 ca prima sursă publică de evenimente de syscall colectate dintr-un scenariu real de honeypot în mediu cloud. Lakha et al. [31] publică în 2022 una dintre cele mai citate lucrări bazate pe BETH, folosind o combinație de Graph Neural Network și Transformer pentru detectarea anomaliilor; rezultatele lor ($AUROC > 0,90$) reprezintă un punct de referință competitiv, însă cu costuri computaționale semnificativ mai mari decât IF. Highnam et al. [26] revin în 2023 cu o lucrare despre proiectarea experimentală adaptivă pentru colectarea de date de intruziune, semnalând că BETH continuă să fie întreținut activ ca resursă academică.

Cea mai apropiată lucrare metodologică de KernelTrap o reprezintă studiul lui Atawneh et al. [13], publicat în 2025 în *Symmetry*, care aplică Isolation Forest direct pe setul BETH și depășește performanțele de referință raportate în lucrarea originală a setului de date. Diferența esențială dintre [13] și KernelTrap este că primul rămâne un studiu de evaluare pe date statice, în timp ce KernelTrap propune un sistem complet de producție. Eremin [21] extinde această linie cu o evaluare a zece algoritmi de învățare pe fluxuri (incluzând variante de IF) pe setul BETH, confirmând fezabilitatea detecției nesupervizate în regim de streaming. Khan et al. [30] explorează o direcție alternativă bazată pe Multi-Agent Reinforcement Learning aplicat tot pe BETH; deși rezultatele lor sunt promițătoare, complexitatea de antrenare și interpretabilitatea redusă fac abordarea improprie pentru un sistem de producție cu cerințe stricte de latență.

În ceea ce privește selecția caracteristicilor pentru ML aplicat la syscall-uri, Mvula et al. [39] compară diferite tehnici de extracție a caracteristicilor (one-hot, n-grame, word embeddings) pe trace-uri de syscall, iar Użupis et al. [48] introduc o metodă de grupare bazată pe risc care îmbunătățește acuratețea SVM cu 23,4%. Aceste rezultate susțin direct alegerea făcută în KernelTrap de a extinde setul clasic de 7 caracteristici BETH cu derivate calculate din câmpurile `args`, `processName`, `timestamp` și `threadId` ignorate de versiunile anterioare ale modelelor BETH, ajungând la un total de 22 de dimensiuni proiectate să echilibreze puterea discriminativă cu costul computațional al evaluării.

LIGHT-HIDS [32] reprezintă o lucrare recentă (2025) deosebit de relevantă, propunând un cadru hibrid Deep SVDD + detecție de noutate pe secvențe de syscall, cu un câștig de viteză la inferență de până la 75 de ori. Lucrarea confirmă fezabilitatea sistemelor ML-HIDS ușoare în producție, dar nu integrează nici răspuns activ, nici componente de honeypot.

3.3 Praguri adaptive și agregare temporală

Două direcții complementare sprijină arhitectura de decizie a KernelTrap. Cui et al. [18] și Cao et al. [16] documentează fundamentarea teoretică a abordării **User and Entity Behavior Analytics (UEBA)**, demonstrând că pragurile per-entitate (calibrate individual pentru fiecare utilizator) reduc semnificativ rata de fals pozitive comparativ cu un prag global unic — o observație implementată direct în KernelTrap prin pragurile de severitate calibrate per-UID pe partiția de validare a setului BETH. Dani et al. [19] formalizează utilizarea pragurilor adaptive bazate pe percentile, justificând alegerea percentilelor 2,0% și 0,2% pentru cele două niveluri de severitate.

Pe direcția agregării temporale, sondajul publicat în *PeerJ Computer Science* [8] sintetizează tehnicile de fereastră glisantă, aplicate la detectarea anomaliilor pe fluxuri de date de securitate. Algoritmul iForestASD [20] reprezintă una dintre primele formulări ale combinației Isolation Forest + fereastră glisantă în context de streaming, iar Yunus et al. [50] extind ideea cu o serie de ferestre paralele care reduc rata de fals pozitive prin votare. Aceste contribuții susțin direct mecanismul de agregare temporală implementat în KernelTrap, care combină un prag absolut (cel puțin 5 evenimente de severitate 2 într-o fereastră de 60 de secunde) cu un prag de rată (cel puțin 30% din totalul evenimentelor utilizatorului în fereastră să fie de severitate 2), reducând astfel rata de fals pozitive pe utilizatori legitimi cu activitate intensă.

3.4 Honeypot-uri SSH cu interacțiune ridicată

Honeypot-ul Cowrie rămâne, în literatura recentă, soluția de referință pentru capturarea atacurilor SSH. Lucrarea Q-Cowrie [9], publicată în 2026 în *International Journal of Information Security*, propune o variantă adaptivă bazată pe Q-learning, demonstrând relevanța continuă a Cowrie ca platformă experimentală. O direcție inovatoare este reprezentată de Sladić et al. [44] în lucrarea *LLM in the Shell* (IEEE EuroS&PW 2024), care integrează modele lingvistice mari peste arhitectura Cowrie pentru a genera răspunsuri shell credibile la comenzi neașteptate ale atacatorului. Reworr și Volkov [41] aplică o abordare similară pentru a monitoriza specific agenți de hacking bazați pe LLM, raportând capturarea unor modele comportamentale distincte față de atacatorii umani.

Pe direcția izolării prin containere, Iwendi et al. [27] documentează un sistem honeypot complet containerizat în cloud, iar HoneyFactory [49] propune o arhitectură honeynet bazată pe containere Docker cu suport pentru orchestrare automată. Studiul comparativ publicat în 2025 în *Informatics* [4] analizează șapte honeypot-uri contemporane și concluzionează că, pentru scenariul SSH cu interacțiune ridicată, Cowrie oferă cel mai bun raport între bogăția datelor capturate și costul operațional. Aceste

lucrări susțin direct alegerea făcută în KernelTrap, unde containerul honeypot expune un mediu Linux real izolat prin Docker, evitând complexitatea unei mașini virtuale dedicate.

O limitare comună a tuturor acestor sisteme este însă natura lor pasivă: atacatorul trebuie să decidă să interacționeze cu honeypot-ul. Atunci când un atacator a compromis deja o gazdă legitimă printr-un cont SSH valid, niciunul dintre honeypot-urile descrise mai sus nu îl poate redirecționa acolo.

3.5 Apărare activă și pivotare transparentă

Lucrarea cea mai apropiată conceptual de mecanismul de pivotare al KernelTrap este *Cyber Deception Reactive: TCP Stealth Redirection to On-Demand Honeypots* a lui Lopez et al. [36], publicată în februarie 2024. Autorii propun un sistem care interceptează la nivel TCP conexiunile atacatorului și le redirecționează discret către un honeypot clonat, fără ca atacatorul să sesizeze tranziția. Diferențele de concepție față de KernelTrap sunt totuși semnificative: sistemul lui Lopez et al. operează pe baza unor reguli statice de detecție la nivel de rețea, în timp ce KernelTrap declanșează pivotarea pe baza unei decizii ML calibrate pe comportamentul individual al utilizatorului, observat la nivel de syscall, nu de pachet de rețea. Mai mult, KernelTrap utilizează semnalul POSIX `SIGUSR1` combinat cu reguli `iptables` pentru a forța reconectarea sesiunii SSH existente, în timp ce abordarea bazată exclusiv pe NAT a lui Lopez et al. nu poate trata sesiunile deja stabilite.

Sondajul publicat în *Computers & Security* [1] (2024) sintetizează cele mai recente tehnici de înșelăciune cibernetică, incluzând redirecționarea, mutarea țintelor (MTD) și deceptia bazată pe AI. Lucrarea identifică redirecționarea reactivă ca pe o direcție emergentă cu puține implementări academice complete, o observație care confirmă noutatea contribuției KernelTrap. Kahlhofer și Rass [28] demonstrează la EuroS&PW 2024 fezabilitatea unei deceptii la nivel aplicativ fără modificarea aplicațiilor protejate, principiu reflectat în KernelTrap prin faptul că redirecționarea atacatorului nu necesită modificări ale daemon-ului SSH sau ale aplicațiilor existente pe gazda compromisă.

În plus, conceptul de **honeypot** în implementarea KernelTrap sub forma fișierelor canary plasate în containerul honeypot este susținut de lucrarea seminală a lui Bourke și Grzelak [14] (Black Hat Asia 2018), care demonstrează empiric o rată de declanșare de 83% a token-urilor AWS scurse pe GitHub, și de sondajul recent al lui Khan et al. [2] privind honeypot-urile în apărarea activă.

3.6 Sisteme integrate similare KernelTrap

Două lucrări academice publicate în 2025 abordează probleme suficient de similare cu KernelTrap pentru a merita o comparație directă, ele reprezentând cele două extremități ale spațiului soluțiilor în care se situează contribuția prezentă.

Atawneh et al. [13] (2025, *Symmetry*) propun un sistem de detectare a intruziunilor bazat pe Isolation Forest antrenat pe setul BETH, integrat cu un honeypot pentru colectarea datelor. Diferențele de design față de KernelTrap sunt următoarele: lucrarea citată folosește honeypot-ul exclusiv ca sursă de date pentru antrenarea modelului, nu ca mecanism de răspuns activ; nu implementează agregare temporală cu fereastră glisantă; nu propune un mecanism de pivotare dinamică; și nu funcționează ca sistem de producție distribuit cu mai mulți agenți. Cu toate acestea, valoarea AUROC raportată (0,87) constituie un punct de referință direct comparabil pentru evaluarea modelului de detecție al KernelTrap.

Satpute et al. [43] (2025, *ICCK Transactions on Cybersecurity*) descriu un sistem de detectare a intruziunilor bazat pe Random Forest și Autoencoder antrenate pe date colectate dintr-un honeypot SSH (Cowrie). Abordarea lor este parțial supervizată folosește etichete binare extrase din log-urile honeypot-ului și nu acoperă scenariul detectării atacatorilor pe sisteme legitime, ci doar analizează atacatori care au ales să se conecteze deja la honeypot. KernelTrap inversează această logică: detectarea are loc pe gazda reală, iar honeypot-ul este destinația pivotării, nu sursa observațiilor primare.

O lucrare aflată la jumătatea drumului dintre cele două este *AI-Powered Intrusion Detection System with Honeypot Integration* [5], care combină detectarea ML cu un honeypot de tip Cowrie într-un cadru integrat, fără însă a aborda mecanismul de pivotare automată. Sistemul rămâne o concatenare de două componente independente, fără bucla de feedback prezentă în KernelTrap.

3.7 Sinteză și poziționarea KernelTrap

Analiza literaturii recente relevă două observații importante. Pe de o parte, fiecare componentă individuală a KernelTrap se sprijină pe lucrări academice solide: monitorizarea prin eBPF/Tracee este validată de [22, 24, 45, 47], detectarea anomaliilor prin Isolation Forest pe BETH este susținută de [13, 21, 25, 33], honeypot-ul cu interacțiune ridicată în stilul Cowrie este analizat de [4, 9, 44], iar pivotarea reactivă a fost introdusă de Lopez et al. [36]. Pe de altă parte, niciuna dintre lucrările examinate nu integrează aceste patru componente într-un sistem unitar. Tabelul 3.1 sintetizează această observație.

Sistemele comerciale precum Falco, Tetragon, Tracee și KubeArmor oferă monitorizare excelentă, dar nu includ nici detecție bazată pe ML, nici componente honeypot,

Tabela 3.1: Comparație între soluțiile existente și KernelTrap pe cele patru dimensiuni cheie

Sistem	eBPF/syscall	ML	Honeypot	Răspuns activ
Falco / Tetragon / Tracee	Da	Nu	Nu	Nu
Atawneh et al. [13]	Da	Da	Parțial	Nu
Satpute et al. [43]	Nu	Da	Da	Nu
AI-Powered IDS [5]	Nu	Da	Da	Nu
Lopez et al. [36]	Nu	Nu	Da	Da
KernelTrap	Da	Da	Da	Da

nici răspuns activ prin pivotare. Lucrările academice apropiate metodologic (Atawneh 2025, Satpute 2025, AI-Powered IDS 2025) tratează ML-HIDS și honeypot-ul ca componente separate, fără bucla de feedback care unifică KernelTrap. Lucrarea lui Lopez et al. demonstrează fezabilitatea pivotării transparente, dar bazată pe reguli statice de rețea, nu pe decizie ML calibrată comportamental.

Contribuția specifică a KernelTrap, pe care prezenta lucrare urmărește să o demonstreze experimental în capitolele următoare, constă în următoarea combinație: (1) un agent eBPF distribuit, fără stare, cu filtrare strictă pe sesiuni SSH active și excluderea propriei activități pentru a evita bucla de auto-întărire, care minimizează zgomotul și suprafața de atac; (2) un model Isolation Forest extins la 22 de caracteristici (incluând derivate din `args`, `processName`, `timestamp` și `threadId`, ignorate de literatura BETH anterioară), cu praguri adaptive per-(gazdă, utilizator) pe două niveluri de severitate, antrenat pe BETH cu hiperparametri ajustați specific scenariului syscall (`n_estimators` = 2000, percentile 2,0% și 0,2%) și complementat de o listă albă de procese de infrastructură pentru atenuarea derivatei de distribuție; (3) un mecanism de agregare prin fereastră glisantă cu prag dublu (cantitativ și de rată) care convertește scoruri zgomotoase în decizii robuste; (4) o pivotare transparentă declanșată automat care combină semnale POSIX, blocare `iptables` a conexiunilor noi și `exec` către un container honeypot Docker cu identitate oglindită, fără reguli NAT și fără reconectarea sesiunii SSH; și (5) un panou de control în timp real care permite și intervenție manuală. Această combinație nu apare în nicio lucrare publicată cunoscută și constituie principala contribuție a tezei de față.

Capitolul 4

Abordarea propusă

Capitolul de față descrie arhitectura și deciziile de proiectare ale sistemului KernelTrap, explicând *de ce* au fost alese soluțiile tehnice prezentate și cum se articulează componentele într-un flux coerent de la monitorizare la răspuns activ. Pornind de la tehnologiile individuale introduse anterior (sisteme HIDS/IPS, eBPF, Isolation Forest, Redis Streams și containere Docker), prezentul capitol tratează modul în care acestea sunt combinate și configurate pentru a forma un sistem inteligent de prevenire a intruziunilor.

KernelTrap este, în esență, un *Host Intrusion Prevention System* (HIPS) distribuit, bazat pe monitorizare comportamentală la nivel de kernel. Nucleul sistemului îl constituie motorul de detecție: un model Isolation Forest antrenat pe date reale de syscall care evaluează în timp real fiecare eveniment generat de sesiunile SSH active și produce un scor de anomalie. Mecanismul de prevenire, adică răspunsul activ, constă în pivotarea transparentă a sesiunii compromise către un container honeypot izolat, fără ca atacatorul să fie alertat.

Abordarea rezolvă o limitare fundamentală a honeypot-urilor tradiționale: un atacator care a compromis deja un sistem legitim nu are niciun motiv să migreze voluntar către un sistem-momeală. KernelTrap elimină această dependență prin detecție automată bazată pe comportament și pivotare inițiată de sistem, nu de atacator.

Așa cum a reieșit din analiza stadiului actual prezentată în capitolul anterior, niciuna dintre soluțiile examinate nu integrează simultan monitorizarea eBPF, detecția prin învățare automată nesupervizată, honeypot-ul cu interacțiune ridicată și mecanismul de răspuns activ. Fiecare decizie de proiectare expusă în secțiunile următoare se sprijină pe rezultate empirice raportate în lucrările citate în capitolul 3, care justifică alegerile făcute și permit poziționarea KernelTrap față de starea curentă a domeniului.

4.1 Vedere de ansamblu

Sistemul KernelTrap este structurat în patru componente funcționale, distribuite pe mai multe mașini fizice sau virtuale și interconectate prin Redis Streams:

1. Agentul de colectare (*agent*) rulează pe fiecare gazdă protejată și captează evenimentele de syscall produse de sesiunile SSH active, normalizează și filtrează datele, după care le publică pe un stream Redis dedicat gazdei. Este componenta de telemetrie a IPS-ului: ușoară, fără stare și fără rol decizional.
2. Serverul central de analiză și decizie (*central server*) reprezintă creierul IPS-ului: agregate fluxurile de la toți agenții, atribuie un scor fiecărui eveniment cu modelul Isolation Forest, aplică mecanismul de fereastră glisantă și decide automat când un utilizator depășește pragul de anomalie. Serverul emite comenzile de răspuns activ pe stream-urile de comandă ale gazdelor afectate și expune un API de tip REST împreună cu o conexiune WebSocket pentru panoul de control.
3. Containerul honeypot (*hp-shell*) este mecanismul de izolare și observare activat după decizia IPS-ului: primește sesiunile SSH pivotate, expune un mediu Linux aparent funcțional populat cu fișiere-capcană (*canary files*), înregistrează integral activitatea atacatorului prin captură de terminal și raportează propriile evenimente de syscall înapoi la serverul central prin același mecanism de streaming.
4. Panoul de control (*dashboard*) oferă operatorilor de securitate o interfață web în timp real pentru monitorizarea agenților conectați, inspecția fluxului de evenimente procesate și declanșarea manuală a pivotărilor, complementând automatizarea cu posibilitatea de intervenție umană.

Arhitectura este deliberat asimetrică: agenții sunt ușori și fără stare, toate deciziile de analiză și răspuns fiind centralizate în serverul central. Aceasta permite adăugarea de noi gazde monitorizate fără modificarea serverului: agentul nou publică pe un stream nou, iar serverul îl descoperă automat. Cuplajul slab dintre componente este asigurat de Redis Streams, care acționează ca magistrală de mesaje persistentă.

Fluxul de date parcurge sistemul în patru faze (Figura 4.1): **Colectare** (agentul captează syscall-urile și le publică) → **Detectie** (serverul central atribuie scoruri evenimentelor și realizează agregarea acestora prin intermediul unei ferestre glisante) → **Decizie IPS** (pragul de anomalie este depășit) → **Răspuns activ** (pivotare în honeypot + blocare IP).



Figura 4.1: Fluxul de date în KernelTrap. Cele patru componente comunică exclusiv prin Redis Streams: agentul publică evenimente de syscall, serverul central le scorează și emite comenzi de pivotare, iar agentul redirectează sesiunea SSH compromisă către containerul honeypot.

4.2 Colectarea și preprocesarea evenimentelor la nivel de agent

Un sistem de monitorizare bazat pe syscall-uri se confruntă imediat cu o problemă de volum: un server Linux tipic generează zeci de mii de syscall-uri pe secundă, provenite din sute de procese, inclusiv daemoni de sistem, sarcini cron și servicii de rețea. Includerea tuturor acestor evenimente în analiza de anomalii ar genera un fond de zgomot care ar diminua sensibilitatea detecției, iar antrenarea modelului pe astfel de date ar conduce la învățarea unor tipare irelevante pentru scenariile de atac vizate.

KernelTrap răspunde acestei probleme prin trei principii de filtrare aplicate în lanț la nivelul agentului:

Filtrarea pe sesiune SSH. Doar evenimentele generate de utilizatorii cu sesiuni SSH active sunt transmise mai departe. Rațiunea este dublă: atacatorii care obțin acces interactiv prin SSH constituie populația-țintă a sistemului, iar procesele de sistem (init, cron, sshd, daemoni de rețea), cu UID-uri sistemice fixe, sunt excluse prin liste albe gestionate atât la agent, cât și pe serverul central.

Excluderea propriei activități a agentului. Tracee captează toate apelurile de sistem de pe gazdă, inclusiv pe cele emise de procesul agentului atunci când publică evenimentele. Fără un mecanism de auto-excludere, syscall-urile agentului ar fi raportate către serverul central, clasificate ca anormale (sunt rare în datele de antrenare) și ar genera o buclă de auto-întărire cu fals pozitive. Agentul își exclude prin urmare propriile evenimente și pe cele ale procesului-părinte, înaintea publicării.

Normalizarea identității syscall-urilor. Tracee raportează evenimentele prin denumiri textuale, pe care KernelTrap le mapează la identificatori numerici consecutivi printr-un tabel de corespondență stabil între rulări și consistent pe toate gazdele monitorizate. Această codificare permite modelului antrenat să generalizeze indiferent de gazda sursă și acoperă atât syscall-urile standard Linux, cât și hook-urile LSM (Linux

Security Modules) folosite de Tracee pentru evenimente cu context de securitate mai bogat.

Alegerea Tracee ca instrument de captură a evenimentelor este susținută de studiul comparativ al lui Syairozi și Arizal [47], care arată că Tracee oferă cea mai mică amprentă pe CPU și cea mai bună acoperire a hook-urilor LSM dintre soluțiile open-source testate (Falco, Tetragon, KubeArmor, Tracee). La nivelul costului de instrumentare, eBPF-PATROL [6] demonstrează că un agent minimal bazat pe interceptare de syscall-uri poate menține un overhead sub 2,5%

4.3 Infrastructura de comunicație prin Redis Streams

Alegerea Redis Streams [40] ca magistrală de mesaje răspunde unei cerințe specifice: evenimentele trebuie să persiste suficient timp pentru a fi procesate de serverul central chiar și în cazul unui restart temporar, dar sistemul nu trebuie să introducă complexitatea operațională a unui broker distribuit complet (Apache Kafka, RabbitMQ). Față de Redis Pub/Sub, care pierde mesajele dacă niciun subscriber nu este activ, Streams persistă intrările și le face disponibile pentru citire ulterioară.

Topologia de comunicație folosește două tipuri de stream-uri, organizate per-gazdă: un stream de evenimente, pe care fiecare agent își publică telemetria, și un stream de comenzi, prin care serverul central transmite agenților ordine de pivotare. Separarea celor două roluri permite ca agenții să fie complet decuplați unii de alții și să fie adăugați la infrastructură fără modificarea serverului. Descoperirea automată a agenților noi se face printr-un mecanism de tip *zero-configuration discovery*: serverul scanează periodic spațiul de chei Redis și începe să consume orice stream nou apărut, fără configurare manuală.

Arhitectura event-driven decuplează producătorii de evenimente (agenții) de consumatorul de analiză (serverul central), permițând fiecărei părți să continue funcționarea independent de disponibilitatea momentană a celeilalte.

4.4 Modelul de detecție a anomaliilor

KernelTrap folosește Isolation Forest [33] antrenat exclusiv pe date benigne, o abordare *one-class* [17] potrivită contextului în care atacurile reale sunt rare și impredictibile, iar etichetarea lor manuală nu este fezabilă. Compromisul abordării nesupervizate este o rată potențial mai ridicată de fals pozitive decât a clasificatorilor binari, atenuată în KernelTrap prin agregarea temporală descrisă în secțiunea următoare.

4.4.1 Caracteristici și antrenare

Implementările clasice ale modelelor pe BETH [13] folosesc doar șapte câmpuri numerice de bază (`processId`, `parentProcessId`, `userId`, `mountNamespace`, `eventId`, `argsNum`, `returnValue`). `KernelTrap` extinde acest set la 22 de dimensiuni prin patru categorii de derivate: codificare prin frecvență a câmpurilor categoriale (`eventId`, `mountNamespace`, `processName`); indicatori binari extrași din câmpul `args` care semnalează căi sensibile (`/etc/shadow`, `.ssh/`), fișiere canary, traversare de directoare, metacaractere shell și adrese IP/URL; categorii de procese (shell-uri, interpretori, unelte de recon, unelte de rețea, helper-e de escaladare); și caracteristici rolling per-proces calculate pe ferestre de 5 secunde (rata de syscall, diversitatea, raportul de eșecuri, intervalul de timp). Câmpurile `processId`, `parentProcessId` și `userId` nu sunt folosite ca features directe, deoarece, fiind identificatori, scalarea lor ca valori continue induce modelul în eroare; sunt păstrate doar ca chei de grupare pentru caracteristicile rolling și calibrarea pragurilor per-entitate.

Modelul este antrenat pe partiția benignă a setului BETH [25], care conține peste 8 milioane de evenimente colectate prin eBPF din honeypot-uri cloud, aceeași tehnologie folosită de agentul `KernelTrap`, ceea ce minimizează discrepanța dintre datele de antrenare și cele de producție. Caracteristicile sunt standardizate înainte de a fi introduse într-un Isolation Forest cu 2000 de arbori, iar antrenarea folosește o tehnică de eșantionare uniformă care păstrează în memorie cel mult 10 milioane de instanțe benigne, suficient pentru acoperirea diversității comportamentelor normale fără a încălca întregul set.

Pe această configurație, modelul atinge $AUROC = 0,969$ și $Average Precision = 0,994$ pe partiția de test BETH. Aceste valori depășesc atât rezultatele de referință din lucrarea originală a setului [25], cât și valoarea de 0,87 raportată de Atawneh et al. [13] pentru un Isolation Forest aplicat direct pe cele 7 caracteristici clasice ale BETH. Lakha et al. [31] ating AUROC peste 0,90 cu o combinație de Graph Neural Network și Transformer, dar cu un cost computațional semnificativ mai mare. Aceste valori constituie reperele folosite pentru evaluarea modelului din `KernelTrap` în capitolele ulterioare.

4.4.2 Praguri și calibrare în producție

Isolation Forest returnează pentru fiecare eveniment un scor continuu prin `decision_function`: valorile pozitive corespund comportamentului normal, cele negative corespund anomaliilor. `KernelTrap` calibrează praguri individuale per-(`hostname`, `userId`) pe partiția de validare BETH: percentila 2% definește pragul severity 1 (anomalie minoră, raportată în panoul de control dar insuficientă pentru pivotare), iar percentila 0,2% definește pragul severity 2 (anomalie majoră, intră în decizia de pivot). Granularitatea per-(host, utilizator) reflectă faptul că același UID poate corespunde

unor utilizatori complet diferiți pe gazde diferite. Folosirea percentilelor fixe ca mecanism de calibrare urmează metoda formalizată de Dani et al. [19].

În deployment, distribuția live a scorurilor pe o stație Linux modernă diferă semnificativ de cea pe care au fost calibrate pragurile (gazde cloud BETH 2021). Pentru a corecta această derivă fără reantrenare, KernelTrap permite suprascrierea pragurilor în producție și aplică o listă albă de procese de infrastructură (daemoni de desktop și sistem) care forțează severitatea la 0 pentru procesele care nu apar în BETH, dar nici nu reprezintă vectori de atac. Un mecanism de raportare a sănătății modelului expune raportul dintre scorurile live și pragurile teoretice, alertând operatorul când acestea nu mai reflectă distribuția reală.

4.5 Agregarea prin fereastră glisantă

Un eveniment izolat cu severitate 2 nu este o dovadă suficientă pentru a declanșa pivotarea. Comportamentul legitim poate genera ocazional syscall-uri cu scoruri extreme: un administrator care execută o operație rară, un script de backup neobișnuit sau o actualizare de sistem pot produce scurte rafale de anomalii minore. Acceptarea unui singur eveniment ca trigger ar genera un număr inacceptabil de fals pozitive și ar pivota utilizatori legitimi.

KernelTrap menține pentru fiecare pereche (`hostname`, `userId`) o colecție cu marcaj temporal a evenimentelor scoruite din ultimele 60 de secunde. Pivotarea se declanșează numai dacă sunt îndeplinite simultan două condiții: cel puțin 5 evenimente de severitate 2 în fereastră (prag absolut, exclude rafalele accidentale) și cel puțin 30% din totalul evenimentelor utilizatorului să fie de severitate 2 (prag de rată, exclude false pozitive pe utilizatori activi). Pragul de rată este esențial: pe o sesiune cu mii de syscall-uri benigne pe minut, 5 evenimente sev-2 izolate reprezintă mai puțin de 0,5% din activitate și nu constituie semnal suficient; pe o sesiune compromisă în care atacatorul execută unelte de recon sau exploatare, peste 30% din syscall-uri pot fi anormale. Combinația celor două praguri este robustă față de ambele extreme și reduce semnificativ rata de fals pozitive față de pragul pur cantitativ.

Lista albă de UID-uri ale serviciilor de sistem (root și daemoni cu UID-uri fixe) garantează că niciun cont critic nu poate fi pivotat din greșeală. Odată ce un utilizator a fost pivotat, starea sa rămâne marcată până la reinițializarea manuală.

Combinația Isolation Forest cu fereastră glisantă a fost validată în literatură de algoritmul iForestASD [20], una dintre primele formulări ale acestui cuplaj în context de streaming. Yunus et al. [50] extind ideea cu mai multe ferestre paralele evaluate prin votare, demonstrând o reducere suplimentară a ratei de fals pozitive un rezultat care confirmă valoarea agregării temporale ca mecanism complementar scoringului ML.

4.6 Mecanismul de pivotare

4.6.1 Detecție și comandă

Când fereastra glisantă depășește ambele praguri, serverul central emite o comandă de pivotare pe stream-ul de comenzi al gazdei afectate. Decizia este transmisă prin UID numeric, nu prin nume de utilizator: într-o infrastructură multi-gazdă, același UID poate corespunde unor utilizatori diferiți pe mașini diferite, iar rezolvarea în nume se face local, pe gazdă. Separarea între decizie (serverul central, fără privilegii) și execuție (agentul local, cu privilegii ridicate) este deliberată: serverul nu necesită acces root pe gazdele monitorizate, ceea ce reduce suprafața de atac a sistemului însuși.

4.6.2 Execuția pivotării

Pivotarea propriu-zisă combină trei mecanisme conceptuale într-o secvență ordonată. *În primul rând*, conexiunile TCP *noi* de la adresa atacatorului sunt blocate la nivel de firewall, în timp ce sesiunea SSH deja stabilită este lăsată intactă; o închidere bruscă a conexiunii ar alerta atacatorul, ratând astfel ocazia de observare. *În al doilea rând*, contul utilizatorului este provizionat dinamic în containerul honeypot, populat cu fișiere-capcană (descrise în secțiunea următoare) și pregătit să accepte sesiunea pivotată. *În al treilea rând*, shell-ul activ al utilizatorului este forțat, printr-un mecanism de semnal POSIX, să-și înlocuiască propriul proces cu o nouă sesiune SSH către containerul honeypot, păstrând conexiunea TCP a clientului. Din perspectiva atacatorului, sesiunea continuă fără întrerupere vizibilă: doar conținutul shell-ului se schimbă, indistinguibil de o conexiune SSH executată voluntar către un alt sistem din rețeaua internă.

Cea mai apropiată lucrare conceptuală este Lopez et al. [36], care propune redirectionare TCP transparentă bazată pe reguli NAT statice. KernelTrap se diferențiază în două aspecte: declanșarea pivotării se face pe baza unei decizii ML calibrate per-(gazdă, utilizator) la nivel de syscall, nu pe reguli statice de rețea; iar mecanismul de înlocuire la nivel de proces shell permite tratarea sesiunilor deja stabilite, fără NAT și fără reconectare, un scenariu pe care abordarea pur TCP a lui Lopez et al. nu îl acoperă. Independența față de modificarea daemon-ului SSH urmează principiul demonstrat de Kahlhofer și Rass [28].

4.7 Honeypot-ul

4.7.1 Oglindire dinamică a identității

Un honeypot eficient nu trebuie să pară suspect. Dacă utilizatorul pivotat ar descoperi că se află pe un alt sistem (hostname diferit, utilizatori absenți, sistem de fișiere gol), ar putea abandona sesiunea, pierzând orice beneficiu de observare. KernelTrap previne această problemă prin *oglindire dinamică a identității*: la momentul pivotării, scriptul de instalare creează în containerul honeypot un cont cu exact același nume de utilizator ca cel de pe gazda reală, transferă cheile SSH publice ale contului original și configurează shell-ul implicit identic.

Hostname-ul containerului poate fi configurat să coincidă cu cel al gazdei reale, eliminând și această indicație vizibilă a tranziției. Mecanismul de high-interaction honeypot, unde atacatorul interacționează cu un sistem Linux real, nu cu o emulare, permite observarea unor tehnici de atac complexe pe care emulările low-interaction nu le-ar putea captura [46]. Alegerea unui container Docker, în detrimentul unei mașini virtuale dedicate, este sprijinită de studiul publicat în *Informatics* [4], care identifică soluțiile bazate pe containere drept cel mai bun raport între bogăția datelor capturate și costul operațional în scenariul SSH cu interacțiune ridicată, și de Iwendi et al. [27], care documentează un sistem honeypot complet containerizat funcționând în cloud.

4.7.2 Fișiere canary

Așa cum s-a prezentat în capitolul de aspecte teoretice, honeytokens-urile sunt artefacte digitale false al căror simplu acces constituie un indicator incontestabil de compromitere. KernelTrap aplică acest principiu în containerul honeypot prin plasarea de fișiere-capcană (*canary files*) în directorul home al utilizatorului pivotat. Fișierele sunt proiectate pe categorii care imită resursele țintite frecvent de atacatorii motivați financiar sau de spionaj: credențiale cloud (chei de acces de tip AWS), chei SSH prezentate ca *deploy key* pentru servere interne, fișiere de variabile de mediu cu credențiale fictive de bază de date și token-uri de API, note interne care referențiază sisteme și servere inexistente menite să ghideze atacatorul mai departe în rețeaua simulată, precum și token-uri de autentificare pentru sisteme CI/CD. Toate sunt construite cu format realist, astfel încât un atacator care le inspectează să nu poată distinge cu ușurință între canary-uri și artefacte legitime.

Accesul la oricare dintre aceste fișiere este monitorizat prin reguli de audit la nivel de kernel: orice tentativă de citire generează imediat un eveniment vizibil atât în jurnalul containerului, cât și pe panoul de control al serverului central. Identificatorii regulilor de audit sunt deliberat ofușcați, pentru a împiedica un atacator care interoghează regulile active să recunoască ușor fișierele-momeală.

Eficacitatea fișierelor canary este susținută empiric de Bourke și Grzelak [14], care raportează o rată de declanșare de 83% pentru token-urile AWS plasate în depozitele GitHub publice, dovadă clară că atacatorii reali investighează sistematic credențialele găsite în sistemele compromise. Sondajul recent al lui Khan et al. [2] confirmă rolul central al honeypot-urilor în strategiile moderne de apărare activă.

4.7.3 Înregistrarea sesiunilor TTY

Containerul honeypot este configurat astfel încât fiecare sesiune SSH primește automat înregistrare integrală a terminalului (*full tty recording*) înainte de pornirea shell-ului interactiv. Înregistrarea capturează atât intrările tastate de atacator, cât și ieșirile vizibile pe ecran: comenzile executate, rezultatele programelor, editările de fișiere, erorile de autentificare și eventualele tentative de escaladare de privilegii.

Această capacitate de înregistrare completă a sesiunii este esențială atât din perspectiva investigației digitale (reconstituind cronologic toate acțiunile atacatorului), cât și din perspectiva analizei de amenințări: tipare de comportament post-compromitere, tehnicile, tacticile și procedurile folosite, precum și țintele urmărite. Fișierele de sesiune sunt păstrate pe un volum persistent, rămânând disponibile pentru analiză ulterioară chiar și după oprirea sau recrearea containerului.

Spre deosebire de jurnalele clasice, care rețin doar comenzile executate, înregistrarea integrală a terminalului păstrează și ieșirile programelor, precum și editările interactive de fișiere, oferind o reconstituire fidelă a sesiunii așa cum a fost ea percepută de atacator. Această fidelitate este valoroasă mai ales atunci când atacatorul folosește editoare în mod text sau interpretoare interactive, ale căror acțiuni nu lasă urme în istoricul shell-ului.

4.7.4 Monitorizarea în container și feedback loop

Containerul honeypot rulează propriul mecanism de monitorizare a syscall-urilor produse în interiorul său, captând atât execuțiile de comenzi, cât și accesele la fișierele-capcană. Evenimentele rezultate sunt publicate pe propriul stream Redis în același format folosit de agenții de pe gazdele reale, ceea ce permite serverului central să le proceseze, să le atribuie un scor de anomalie și să le afișeze în panoul de control fără logică suplimentară de tratare a sursei.

Această buclă de feedback închide ciclul de observare al IPS-ului: KernelTrap monitorizează comportamentul utilizatorilor atât înainte de pivotare, pe gazda reală, cât și după, în containerul honeypot, menținând o perspectivă continuă asupra sesiunii și posibilitatea de a detecta anomalii suplimentare chiar și în mediul de confinement.

Faptul că evenimentele containerului folosesc exact același format ca cele ale agenților de pe gazdele reale înseamnă că serverul central nu are nevoie de o cale de tratare sepa-

rată pentru sursa lor: aceleași etape de scoring și agregare se aplică uniform, indiferent dacă evenimentul provine de pe o gazdă protejată sau din interiorul honeypot-ului. Astfel, un atacator care continuă să execute comenzi de recon sau de escaladare după pivotare rămâne sub observație, iar comportamentul său din mediul de confinement poate fi comparat direct cu cel dinaintea pivotării.

4.8 Panoul de control în timp real

Panoul de control oferă operatorilor o interfață web unificată pentru monitorizarea și controlul sistemului. Arhitectura de comunicație combină două mecanisme complementare, alese în funcție de natura datelor afișate. **Push în timp real** (WebSocket): evenimentele procesate de serverul central sunt transmise imediat către toți clienții conectați, garantând că orice anomalie majoră devine vizibilă în câteva milisecunde. **Polling periodic** pentru date cu rată de schimbare mai mică: lista agenților conectați, starea ferestrelor glisante per-utilizator și istoricul pivotărilor sunt reîmprospătate la intervale fixe, evitând supraîncărcarea conexiunii principale cu informații care se schimbă rar. Reziliența la întreruperi este asigurată printr-un mecanism de reconectare automată cu interval crescător, astfel încât panoul să tolereze restarturile serverului fără intervenție manuală.

În plus, panoul expune posibilitatea de declanșare manuală a pivotărilor, completând detecția automată cu intervenția umană bazată pe judecata operatorului. Ambele tipuri de pivotare (automată și manuală) sunt înregistrate în istoricul de audit al sistemului, împreună cu tipul de trigger și momentul evenimentului.

Capitolul 5

Aplicația

Capitolul de față descrie implementarea propriu-zisă a sistemului KernelTrap. Structura urmează etapele clasice de dezvoltare software: enunțarea cerințelor funcționale și nefuncționale, analiza pe scurt a problemei, proiectarea arhitecturii și a fluxurilor cheie prin diagrame, descrierea condensată a implementării pe componente și, în final, modul în care a fost verificat sistemul.

5.1 Cerințe

Funcționalitățile vizate au fost transpuse în cerințe specifice, prezentate în tabelele 5.1 și 5.2. Acestea constituie referința în raport cu care sistemul a fost evaluat în capitolul 6.3.

Tabela 5.1: Cerințe funcționale

ID	Descriere
FR1	Agentul colectează în timp real evenimentele de syscall produse pe gazdă și le publică pe serverul central, filtrând automat sesiunile SSH active.
FR2	Serverul central atribuie un scor fiecărui eveniment printr-un model Isolation Forest pre-antrenat și calibrat per-(gazdă, utilizator).
FR3	Sistemul declanșează automat pivotarea unei sesiuni SSH suspecte către containerul honeypot, fără ca atacatorul să sesizeze tranziția.
FR4	Operatorul poate inspecta în timp real activitatea agenților și a utilizatorilor printr-un panou de control web.
FR5	Operatorul poate declanșa manual pivotarea unui utilizator dintr-un buton din panou.
FR6	Containerul honeypot înregistrează integral sesiunea atacatorului și raportează evenimentele înapoi în panou.

Tabela 5.2: Cerințe nefuncționale

ID	Descriere
NF1	Overhead la nivelul gazdei monitorizate sub 2,5% din CPU, măsurat în condiții de încărcare medie.
NF2	Latența între producerea evenimentului declanșator și emiterea comenzii de pivotare sub o secundă.
NF3	Sistemul tolerează căderi temporare ale serverului central; agenții continuă să publice și serverul reia procesarea fără pierdere de evenimente.
NF4	Adăugarea unei gazde noi nu necesită nicio modificare a serverului central (zero-configuration discovery).

5.2 Analiză

Cerințele enunțate mai sus împart sistemul în patru responsabilități clar separate: *colectarea datelor* (FR1), *analiza* și *decizia* de pivotare (FR2, FR3), *izolarea atacatorului* (FR3, FR6) și *observabilitatea* pentru operator (FR4, FR5). Combinate cu cerințele de performanță și de decuplare (NF1–NF4), aceste responsabilități exclud varianta unei aplicații monolitice și favorizează o arhitectură distribuită orientată pe evenimente, în care componentele comunică prin mesaje păstrate într-o magistrală comună, nu prin apeluri sincrone directe. Motivele detaliate ale acestei alegeri, împreună cu justificarea fiecărei tehnologii folosite, au fost expuse în capitolul anterior (Abordarea propusă); secțiunile care urmează tratează modul concret în care arhitectura este pusă în practică.

5.3 Proiectare

Această secțiune prezintă structural sistemul prin trei figuri: arhitectura de componente (figura 5.1), fluxul de detecție și pivotare (figura 5.2) și distribuția scorurilor de normalitate produse de model (figura 5.3).

Arhitectura de componente. Sistemul este organizat în trei zone funcționale, conectate exclusiv prin magistrala Redis Streams: gazda monitorizată (unde rulează agentul și sursa de telemetrie Tracee), serverul central (cu modulul de atribuire a scorului, fereastra glisantă și interfața de control) și containerul honeypot (cu propriul mecanism de monitorizare). Săgețile din figura 5.1 indică direcția și mijlocul de comunicare folosit între componente: scrierea și citirea evenimentelor pe stream-urile Redis (operațiile `XADD` și `XREAD`), apelurile HTTP și conexiunea WebSocket dinspre serverul FastAPI către panoul de control, iar la nivel local pe gazdă semnalul `SIGUSR1` (folosit pentru a declanșa trap-ul de pivotare din shell-ul atacatorului) și regulile `iptables` (pentru blocarea conexiunilor de la IP-ul atacatorului).

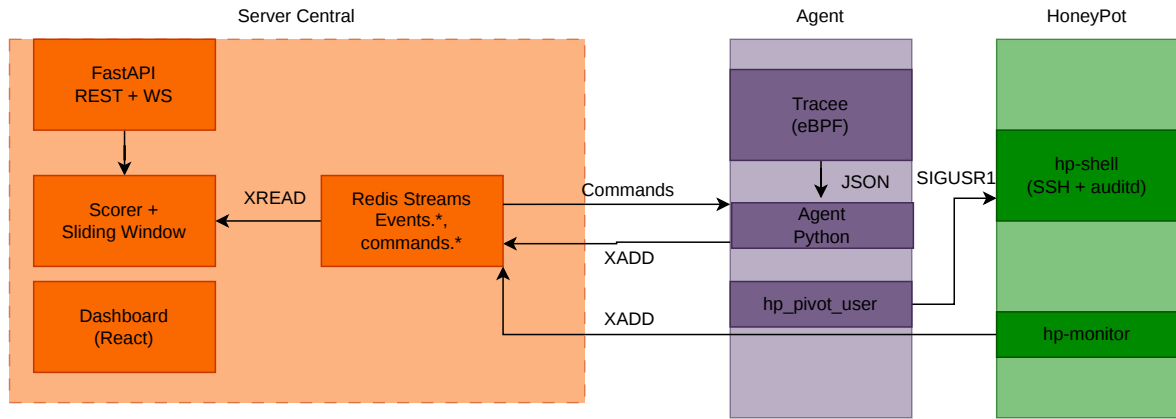


Figura 5.1: Arhitectura de componente. Cele trei zone comunică exclusiv prin Redis Streams; serverul central agregă fluxurile, atribuie scoruri și emite comenzi de pivotare.

Fluxul de pivotare. Diagrama de secvență din figura 5.2 urmărește calea completă a unui eveniment, de la momentul în care Tracee îl captează pe gazdă, până la momentul în care shell-ul atacatorului este înlocuit cu o sesiune SSH către container. Sunt evidențiate patru tranziții cheie: publicarea evenimentelor în loturi pe magistrală, atribuirea scorurilor și agregarea în fereastra glisantă, emiterea comenzii de pivotare către gazda afectată și, în final, execuția pe gazdă a scriptului de pivotare, care realizează în paralel trei sub-acțiuni: blocarea conexiunilor cu `iptables`, provizionarea contului oglindit în container și trimiterea semnalului `SIGUSR1` care comută efectiv sesiunea.

Distribuția scorurilor de normalitate. Pentru fiecare eveniment, modelul Isolation Forest emite un scor continuu: valorile pozitive corespund comportamentului considerat normal, iar cele negative anomaliilor (cu cât scorul este mai aproape de -1 , cu atât evenimentul este mai izolat de restul datelor benigne văzute la antrenare). Figura 5.3 ilustrează distribuția acestor scoruri pe partiția de calibrare, cu cele două praguri marcate vertical: percentila 2,0% ($s = -0,022$, severitate 1) și percentila 0,2% ($s = -0,102$, severitate 2). Cele trei zone rezultate (benign, minor și major) corespund direct nivelurilor de severitate pe baza cărora fereastra glisantă cumulează indicii și decide când să declanșeze pivotarea.

5.4 Implementare

5.4.1 Tehnologii și organizarea proiectului

Sistemul este împărțit în patru componente software, fiecare adaptată rolului pe care îl ocupă (tabelul 5.3). Alegerile principale sunt motivate de cerințele nefuncționale: framework-ul web FastAPI împreună cu serverul `uvicorn` oferă concomitent un API REST și un canal WebSocket asincron pe aceeași conexiune; biblioteca `scikit-learn` pune la dispoziție o implementare matură și bine optimizată a algoritmului Isolation

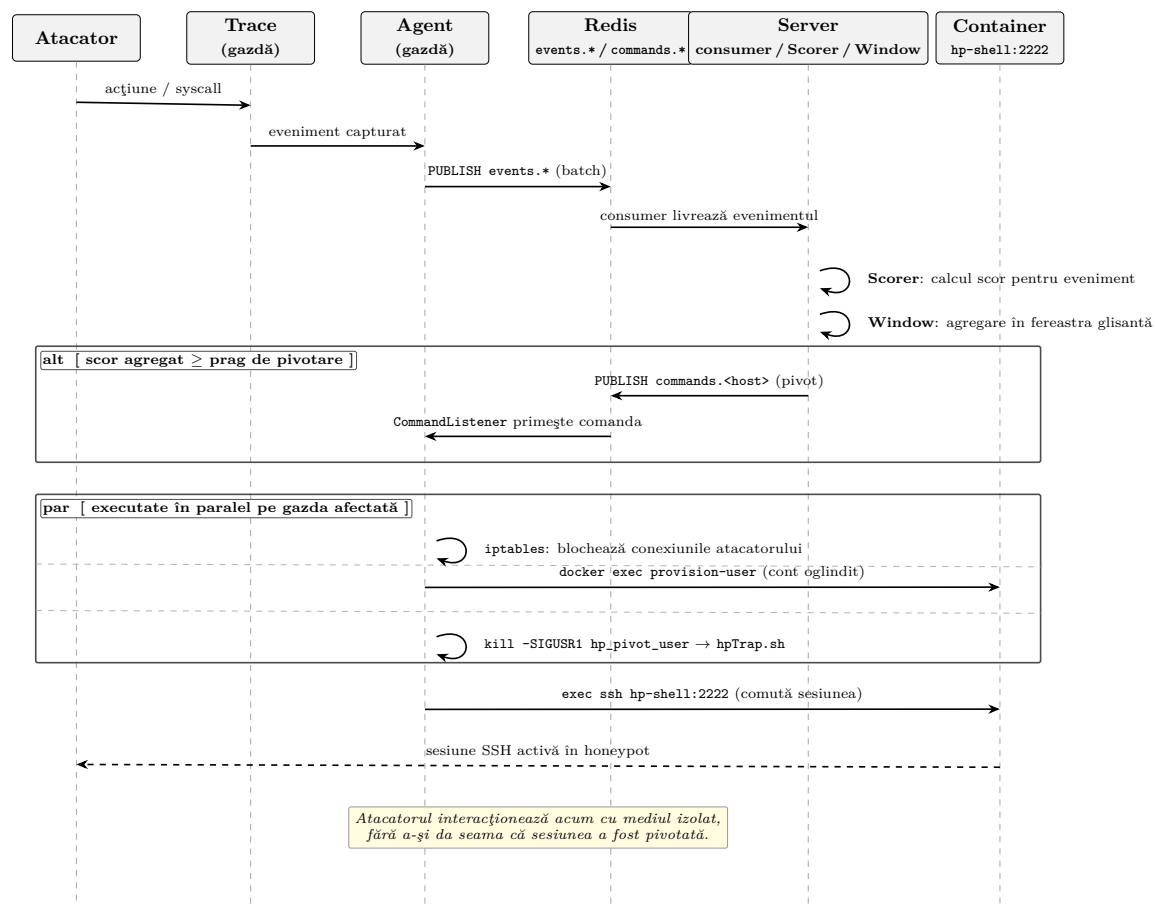


Figura 5.2: Diagrama de secvență a procesului de detecție și pivotare

Forest; Tracee permite instrumentarea kernelului prin eBPF fără să fie nevoie de recompilarea pe fiecare versiune de kernel țintă, iar Redis Streams oferă o magistrală de mesaje persistentă, suficient de robustă pentru cazurile de utilizare ale sistemului, fără a impune complexitatea operațională a unui sistem dedicat de mesagerie de tip Kafka sau RabbitMQ.

Tabela 5.3: Stiva tehnologică pe componente

Componentă	Tehnologii
Agent de colectare	Python 3, Tracee (<code>aquasec/tracee:latest</code>), <code>redis-py</code>
Server central	Python 3, FastAPI, uvicorn, scikit-learn, NumPy
Container honeypot	Docker (Ubuntu 22.04), OpenSSH, <code>auditd</code> , <code>util-linux</code>
Panou de control	React 19, Vite 5, WebSocket
Mesagerie	Redis 7 (Streams)
Container runtime	Docker (<code>docker.io</code>)

Repository-ul este organizat pe componente, cu două scripturi de instalare care ascund detaliile de inițializare:

```
KernelTrap/
```

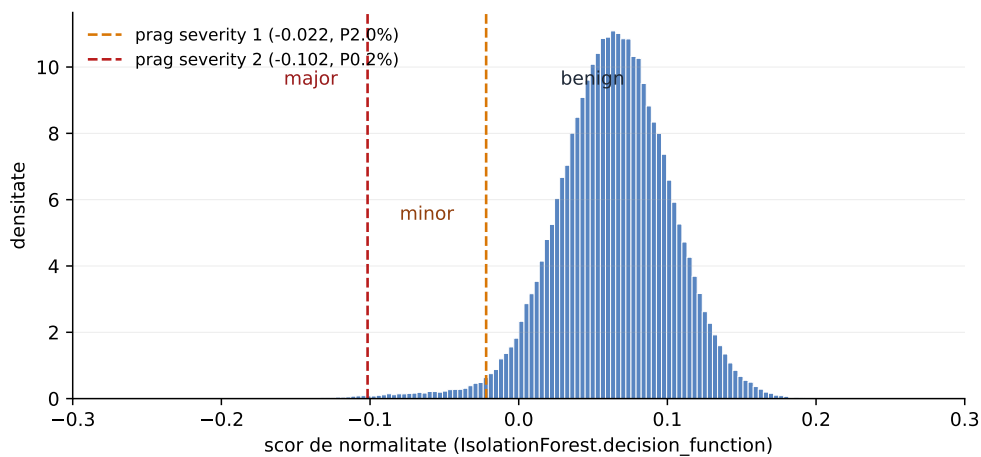



Figura 5.3: Distribuția ilustrativă a scorurilor de normalitate produse de IsolationForest, cu pragurile de severitate 1 și 2 marcate.

```

central_server/          # Serverul central FastAPI + scorer +
    window
    main.py, scorer.py, window_v3.py, requirements.txt
masina_invata/
    isolation_forest/    # Antrenare Isolation Forest pe BETH
    logger/              # Agentul (syscall_logger.py)
HoneyPot/                # Imagine Docker honeypot + script
    pivotare
    Dockerfile, hp_pivot_user.sh, hpTrap.sh
    provision-user.sh, record-session.sh, start-sshd.sh
    hp-monitor.py, install.sh
dashboard/               # Aplicatie React + Vite (panoul de
    control)
start_server.sh          # Bootstrap server central + Redis +
    dashboard
start_agent.sh           # Bootstrap agent pe gazda monitorizata

```

Listing 5.1: Structura repository-ului (esențială)

5.4.2 Componente

Agentul este componenta care rulează pe fiecare gazdă protejată. Primește în timp real evenimentele de syscall capturate de Tracee, le aduce într-un format uniform (inclusiv conversia numelor de syscall în identificatori numerici stabili, pentru a putea fi tratate consistent de modelul de învățare automată) și le trimite serverului central în loturi, prin magistrala Redis. Pentru a reduce zgomotul, păstrează doar evenimentele provenite de la utilizatori conectați activ prin SSH și elimină propriile sale apeluri. Un al doilea fir de execuție așteaptă în paralel comenzi de la server: când primește o decizie de pivotare, lansează scriptul corespunzător pentru a redirecționa sesiunea ata-

catorului în honeypot. La nivel de implementare, toată această logică este concentrată în `masina_invata/logger/syscall_logger.py`, cu clasele `TraceeParser` (parsing) și `CommandListener` (firul de ascultare a comenzilor), folosind biblioteca `redis-py` pentru comunicația cu magistrala.

Serverul central este creierul sistemului: agregă fluxurile de la toți agenții, atribuie un scor fiecărui eveniment și decide când o sesiune trebuie pivotată. Activitatea este împărțită în patru sarcini care rulează în paralel: prima consumă evenimentele primite de la agenți, a doua le atribuie un scor și actualizează fereastra glisantă, a treia publică deciziile de pivotare înapoi către gazde, iar ultima menține panoul de control la zi prin WebSocket. Modulele care implementează aceste responsabilități sunt `central_server/main.py` (orchestrarea celor patru sarcini asincrone peste un server FastAPI), `central_server/scorer.py` (clasa `Scorer`, care încapsulează modelul Isolation Forest împreună cu pipeline-ul celor 22 de caracteristici și lexicoanele de procese cunoscute) și `central_server/window_v3.py` (clasa `SlidingWindowTracker`, care aplică pragul dublu cantitate + rată). Modelul antrenat salvat în trei fișiere (`iforest.joblib`, `scaler.joblib`, `meta.json`) este încărcat în memorie o singură dată la pornire, dintr-un director versionat împreună cu sursa.

Mecanismul de pivotare traduce o decizie a serverului în acțiunea concretă de redirectionare a sesiunii atacatorului. Este realizat din două scripturi shell complementare. Primul, `hp_pivot_user.sh` (instalat în `/usr/local/sbin` cu o regulă `sudoers` care permite agentului să-l ruleze fără parolă), execută în ordine trei acțiuni: blochează conexiunile noi de la IP-ul atacatorului printr-o regulă `iptables`, provizionează contul corespunzător în containerul honeypot prin `docker exec provision-user.sh`, apoi trimite semnalul `SIGUSR1` către shell-urile active ale utilizatorului. Al doilea, `hpTrap.sh`, este livrat ca fragment din `/etc/profile.d/` și se încarcă automat în fiecare shell interactiv la login; instalează un *trap* care, la primirea semnalului `SIGUSR1`, înlocuiește procesul curent al shell-ului cu o sesiune SSH către container, fără ca utilizatorul să observe tranziția:

```
hptrap_pivot() {
    logger -t hptrap "Pivoting user $USER from host to honeypot"
    exec ssh -i /etc/hptrap/hp_key -o StrictHostKeyChecking=no \
        "${USER}@127.0.0.1" -p 2222
}
trap hptrap_pivot USR1
```

Listing 5.2: Trap-ul instalat în fiecare shell interactiv (`hpTrap.sh`)

Containerul honeypot oferă un mediu Linux aparent obișnuit, în care atacatorul ajunge după pivotare. Imaginea de bază este `ubuntu:22.04` cu server SSH, auditare la nivel de kernel (`auditd`) și utilitarele uzuale de linie de comandă. La momentul pivotării, un script de provizionare creează automat în container un cont oglindit (cu

același nume ca pe gazda reală) și populează directorul home al acestuia cu fișiere-momeală plauzibile: chei AWS, o cheie SSH privată, un fișier de mediu cu credențiale fictive de bază de date, note interne și un token CI/CD. Toate aceste fișiere sunt urmărite de `auditd`: orice citire sau scriere asupra lor generează imediat un eveniment. Pe lângă canary-uri, sesiunea integrală a atacatorului este înregistrată ca tranzit TTY, iar un script de monitorizare (`hp-monitor.py`) citește continuu log-ul `auditd` și publică evenimentele înapoi pe magistrala Redis, în același format folosit de agenții de pe gazdele reale, astfel încât panoul de control le poate afișa unitar, indiferent de sursă.

Panoul de control este o aplicație web scrisă în React, păstrată deliberat minimală (singurele dependențe runtime sunt `react` și `react-dom`). Comunică cu serverul central pe două căi: un API REST pentru date relativ statice (lista agenților, istoric pivotări) și o conexiune WebSocket pentru fluxul live de evenimente. Reconectarea automată cu *backoff* exponențial (creșterea progresivă a intervalului dintre încercări) este gestionată centralizat printr-un singur *hook* custom, astfel încât celelalte componente pot presupune că au date la zi. Vizual, panoul este împărțit în patru zone: lista agenților conectați (`AgentsPanel`), fluxul de log-uri în timp real (`LiveLogs`), tabelul utilizatorilor monitorizați cu progresul în fereastra glisantă (`UsersTable`) și istoricul pivotărilor (`PivotHistory`). Aplicația este compilată ca site static și servită direct de serverul FastAPI, eliminând nevoia unui server web separat.

Antrenarea modelului se realizează offline, înainte de punerea în funcțiune a sistemului, pe partiția benignă a setului de date BETH. Pentru a putea folosi întregul volum de date disponibil (zeci de gigabytes), fără a-l încărca complet în memorie, se folosește o tehnică de eșantionare numită *reservoir sampling*: pe măsură ce datele sunt citite în *streaming*, un eșantion uniform de dimensiune fixă este menținut și actualizat probabilistic. Concret, modelul folosit în producție este antrenat pe 10 milioane de instanțe astfel eșantionate, cu o pădure de 2000 de arbori de izolare. Pragurile de severitate 1 și 2 nu sunt fixate arbitrar, ci sunt calibrate ulterior pe o partiție de validare, alegând percentilele 2,0% și 0,2% ale scorurilor de normalitate, astfel încât proporția de evenimente marcate ca anomalii pe date benigne să fie controlată explicit. Aceste praguri, împreună cu tabelele de codificare prin frecvență ale câmpurilor categoriale, sunt salvate în `meta.json` și citite la pornirea serverului central. Scriptul de antrenare se află în `masina_invata/isolation_forest/beth_iforest.py`.

5.5 Testare

Sistemul a fost validat prin cinci tipuri complementare de testare, descrise pe scurt mai jos. Rezultatele cantitative în special cele referitoare la detecția scenariilor reale de atac sunt prezentate detaliat în capitolul 6.3.

Testarea modelului de învățare automată. Scriptul de antrenare expune o

subcomandă dedicată de evaluare care încarcă modelul deja antrenat împreună cu partiția de test a setului BETH, calculează scorurile, aplică pragurile de severitate calibrate și raportează două metrici de referință asupra atacurilor etichetate: aria de sub curba ROC (`roc_auc_evil`) și precizia medie (`avg_precision_evil`). Acest test este reluat la fiecare reantrenare, ca o regresie de calitate care semnalează imediat dacă o nouă versiune a modelului pierde din capacitatea de detecție.

Teste unitare pe componentele critice ale serverului. Cele două clase cu logică algoritmică sensibilă `Scorer` și `SlidingWindowTracker` sunt acoperite de teste unitare `pytest` (`tests/test_scorer.py` și `tests/test_window.py`). Pentru `Scorer` sunt verificate încărcarea corectă a modelului, schema evenimentelor procesate și comportamentul pe input degenerat (lot gol). Pentru `SlidingWindowTracker` sunt verificate condițiile fundamentale ale pragului dublu: nici un pivot sub prag, pivotare exact când cantitatea și rata sunt simultan satisfăcute, blocarea pivotării când rata este sub limită (chiar dacă numărul de evenimente severe e mare), ignorarea UID-urilor omise și suprimarea pivotărilor repetate imediat după un pivot deja declanșat.

Testare de integrare end-to-end. Cea mai relevantă verificare practică se face prin rulări controlate ale întregului sistem: pornirea simultană a serverului central (`start_server.sh`) și a agentului pe o gazdă de test (`start_agent.sh`), generarea manuală de activitate (autentificare SSH, comenzi controlate de recon) și observarea în panoul de control că evenimentele apar, primesc scor, fereastra glisantă cumulează severitățile și, la pragul atins, pivotarea este efectiv declanșată. O singură astfel de rulare exersează magistrala Redis Streams în ambele direcții, scoring-ul, fereastra glisantă și mecanismul de pivotare.

Testarea mecanismului de pivotare și a honeypot-ului. Pivotarea este verificată suplimentar prin invocare directă din linia de comandă: `sudo hp_pivot_user <username>` într-o sesiune SSH activă. Se confirmă apariția regulii `iptables` (vizibilă cu `iptables -L INPUT -n`), tranziția shell-ului către container (PID-ul shell-ului inițial este înlocuit) și prezența fișierelor-momeală în directorul `home` oglindit. Înregistrarea integrală a sesiunii este verificată prin inspecția fișierelor din `/var/log/sessions/` ale containerului, iar regulile de auditare sunt validate prin acces deliberat la fișierele-capcană și observarea evenimentelor corespunzătoare în panou.

Testarea panoului de control. Interfața web este verificată manual în browser: indicatorul de status al serverului (verde/roșu), fluxul de evenimente prin WebSocket, tabelul utilizatorilor cu progresul ferestrei glisante și declanșarea unui pivot prin butonul dedicat. Reconectarea automată cu *backoff* exponențial este verificată prin oprirea voluntară a serverului în timpul rulării panoului clientul se reconectează la repornirea serverului, fără intervenție manuală.

Această combinație a fost preferată unei suite extinse de teste unitare pe toate componentele: întrucât sistemul este distribuit și comunică exclusiv prin Redis Streams,

comportamentul realist apare doar la integrare. Testele unitare au fost menținute strict acolo unde logica este suficient de izolată și de critică pentru a justifica izolarea în Scorer și în fereastra glisantă iar restul componentelor sunt acoperite de testele de integrare end-to-end.

Capitolul 6

Evaluare

Capitolul de față prezintă evaluarea experimentală a sistemului KernelTrap. Sunt enunțate, mai întâi, asumările de securitate pe care se sprijină funcționarea corectă a sistemului; sunt apoi raportate metricile de performanță măsurate pe mediul de testare, urmate de trei scenarii concrete de atac executate cu unelte folosite curent în comunitatea de securitate ofensivă; secțiunea finală tratează limitările sistemului și clasele de atacatori pe care îi acoperă cu eficacitate maximă.

6.1 Asumări de securitate

KernelTrap este un sistem de prevenire a intruziunilor proiectat pentru un scenariu specific: un cont SSH neprivilegiat compromis pe o gazdă altfel necompromisă. Validitatea protecției oferite depinde de două asumări fundamentale asupra mediului de rulare, sintetizate în tabelul 6.1.

Tabela 6.1: Asumările de securitate ale sistemului

Asumare	Consecință
Contul <code>root</code> nu este compromis.	Agentul, scriptul de pivotare, regulile <code>iptables</code> și controlul containerului honeypot rulează cu privilegii ridicate. Un atacator care a obținut deja drepturi <code>root</code> poate dezactiva agentul, șterge regulile de firewall, opri procesul de scoring sau modifica însăși logica de pivotare.
Atacatorul nu are acces fizic la mașină.	Atacuri prin consola fizică (<code>tty*</code>), dispozitive USB ostile, modificare hardware, reset GRUB sau extragere de disc cad în afara domeniului de aplicabilitate. Filtrarea agentului pe sesiuni SSH <code>pts/*</code> reflectă explicit această alegere.

Din aceste asumări derivă două consecințe operaționale: telemetria colectată prin eBPF/Tracee este considerată de încredere atâta timp cât kernelul însuși nu este

compromis; iar scriptul de pivotare, deși expus pentru a putea fi rulat fără parolă de către agent (printr-o regulă `sudoers` de tip `NOPASSWD` aplicată strict pe comanda `hp_pivot_user`), nu este proiectat să reziste unui atacator care a obținut deja drepturi de modificare a fișierelor de sistem.

6.2 Mediul de testare și metodologie

Configurația folosită pentru evaluare este descrisă în tabelul 6.2. Topologia a constat dintr-o gazdă monitorizată, serverul central de analiză și containerul honeypot, rulate pe aceeași subrețea locală pentru a izola măsurătorile de variabilitatea WAN.

Tabela 6.2: Configurația mediului de testare

Componentă	Configurație
Gazda monitorizată	Debian GNU/Linux 12 (bookworm), kernel 6.1.0-45-amd64, procesor virtual KVM (4 nuclee, 4 thread-uri), 4,1 GB RAM
Serverul central	Debian GNU/Linux 12 (bookworm), kernel 6.1.0-45-amd64, AMD Ryzen 9 5950X (24 nuclee fizice / 24 thread-uri), 42 GB RAM
Container honeypot	Docker (Ubuntu 22.04), capabilități <code>AUDIT_WRITE</code> , <code>AUDIT_CONTROL</code>
Versiune Tracee	<code>aquasec/tracee:latest</code> (cfbbfee972e6)
Versiune Python	3.11.2
Versiune Redis	7 (image oficială Docker)

Evaluarea a vizat două categorii de proprietăți: (1) overhead-ul de performanță introdus pe gazda monitorizată, în repaus și în regim de lucru; (2) capacitatea de detecție pe trei scenarii cu unelte publice de atac.

6.3 Performanță

Tabelul 6.3 sintetizează metricile măsurate pe mediul descris mai sus, raportate la țintele nefuncționale enunțate în capitolul 5 (tabelul 5.2); overhead-ul pe CPU al agentului, principala cerință nefuncțională, este ilustrat în Figura 6.1.

6.4 Scenarii de atac și răspunsul sistemului

Pentru a verifica capacitatea de detecție pe acțiuni cu profil realist, au fost executate trei scenarii pe gazda monitorizată, după autentificarea unui cont SSH neprivilegiat. Fiecare scenariu este descris prin succesiunea de acțiuni ale atacatorului, indicatorii observați de KernelTrap și verdictul sistemului.

Tabela 6.3: Metrici de performanță

Metrică	Valoare	Țintă	Verdict
CPU agent (idle)	0,3 %	< 2,5% (NF1)	Atins
CPU agent (în utilizare)	1,8 %	< 2,5% (NF1)	Atins
RAM agent	10,3 MB	—	—
RAM server central	2,54 GB	—	—
Throughput susținut server	~ 6 600 ev/s	—	—

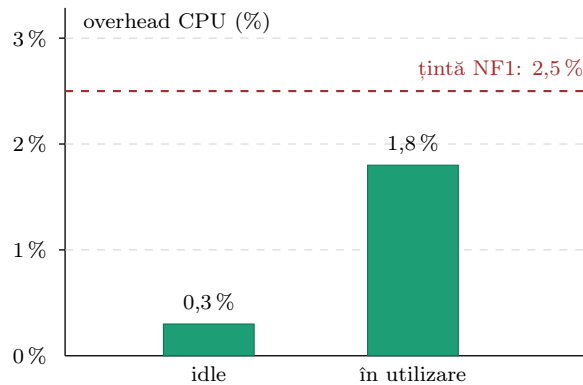


Figura 6.1: Overhead-ul pe CPU al agentului de colectare în repaus (idle) și în utilizare, raportat la ținta nefuncțională NF1 de sub 2,5%. Ambele valori măsurate rămân sub prag.

6.4.1 Scenariul 1: recon automat cu LinPEAS

LinPEAS este un script Bash open-source folosit pe scară largă pentru enumerarea automată a căilor de privilegie escalation pe sisteme Linux. La execuție lansează în cascadă zeci de procese pentru colectarea de informații despre sistem, conturi, configurări de servicii și permisiuni.

Tabela 6.4: Scenariul 1: indicatori observați la rularea LinPEAS

Acțiunea atacatorului	Indicator KernelTrap
wget/curl script de pe host extern	is_network_tool=1, has_url_or_ip=1
id, whoami, uname, hostname, ss, ps în cascadă	is_basic_recon=1 repetat
Citire /etc/passwd, /etc/shadow, .ssh/	has_sensitive_path=1
Rafale dense de syscall-uri	syscall_rate_5s și unique_syscalls_5s ridicate

Rezultat: încă din primele secunde de execuție, severitatea 2 apare pe majoritatea evenimentelor generate de LinPEAS, întrucât rafalele dense de procese de recon combină simultan mai mulți indicatori anormali, anume unelte de rețea, citirea căilor sensibile și o rată ridicată de syscall-uri. Pe măsură ce scriptul rulează, fereastra gli-

santă asociată perechii (gazdă, utilizator) acumulează rapid evenimente de severitate 2, iar proporția acestora în totalul activității utilizatorului depășește pragul de 30%, deoarece sesiunea nu conține activitate benignă semnificativă care să dilueze semnalul. În momentul în care ambele condiții ale pragului dublu sunt îndeplinite simultan, cel puțin 5 evenimente de severitate 2 și o rată de peste 30%, serverul central emite comanda de pivotare. Întreaga secvență, de la lansarea LinPEAS până la redirectionarea sesiunii, devine vizibilă în panoul de control sub forma evenimentelor scoruite și a intrării corespunzătoare în istoricul pivotărilor.

6.4.2 Scenariul 2: stabilirea unui canal C2 cu Sliver

Sliver este un framework C2 open-source modern, similar conceptual cu Cobalt Strike, folosit în testele de penetrare și în campanii reale de post-exploatare. Scenariul a constat din descărcarea implantului Sliver pe gazda compromisă, marcarea executabilă și pornirea acestuia; implantul a deschis o conexiune outbound persistentă către serverul C2, iar operatorul a emis remote comenzi standard de recon.

Tabela 6.5: Scenariul 2: indicatori observați la setup Sliver

Acțiunea atacatorului	Indicator KernelTrap
Fetch implant (<code>curl/wget</code>)	<code>is_network_tool=1, has_url_or_ip=1</code>
Pornire implant cu nume custom	<code>processName_freq=0</code> (necunoscut datele BETH)
Conexiune persistentă către C2 (<code>connect, sendto, recvfrom</code>)	<code>syscall_rate_5s</code> susținut pe socket non-standard
Comenzi remote: <code>id, sudo -l, whoami</code>	<code>is_basic_recon=1, is_privileged_helper=1</code>

Rezultat: deși Sliver generează un volum de `syscall`-uri mai redus decât reconul zgomotos al LinPEAS, combinația de indicatori rămâne suficient de distinctivă. Numele de proces al implantului, absent din datele de antrenare BETH, primește o frecvență zero și contribuie la scoruri mari de anomalie; la acesta se adaugă conexiunea persistentă către serverul C2 și comenzile de recon executate la distanță de operator. Acumularea acestor evenimente de severitate 2 împinge fereastra glisantă peste ambele praguri, iar pivotarea automată redirectionează sesiunea SSH activă către containerul honeypot. În paralel, regula `iptables` adăugată pe adresa serverului C2 blochează stabilirea oricăror conexiuni noi de la același atacator, fără a întrerupe conexiunea deja existentă, care este preluată de honeypot pentru observare continuă.

6.4.3 Scenariul 3: acces la fișiere canary post-pivotare

După redirecționarea în honeypot, atacatorul (sau implantul automat al acestuia) explorează directorul home al utilizatorului oglindit și încearcă să citească fișiere ce par să conțină credențiale: chei AWS (`.aws/credentials`), cheie privată SSH (`.ssh/id_rsa`), fișier de mediu cu credențiale de bază de date (`.env`), note interne (`notes.txt`) și un token de CI/CD (`.jenkins_token`). Pentru fiecare astfel de acces, regulile `auditd` instalate de `provision-user.sh` emit câte un eveniment, captat de `hp-monitor.py` și publicat pe stream-ul Redis al containerului.

Tabela 6.6: Scenariul 3: indicatori observați la accesul canary

Fișier canary accesat	Indicator KernelTrap
<code>~/.aws/credentials</code>	<code>has_canary_path=1</code> , regulă <code>audit_001</code>
<code>~/.ssh/id_rsa</code>	<code>has_canary_path=1</code> , regulă <code>audit_002</code>
<code>~/.env</code>	<code>has_canary_path=1</code> , regulă <code>audit_003</code>
<code>~/notes.txt</code>	regulă <code>audit_004</code>

Rezultat: spre deosebire de detecția probabilistică din scenariile precedente, în care decizia se bazează pe acumularea de scoruri de anomalie, accesul la un fișier canary constituie un indicator binar de compromitere. Niciun utilizator legitim nu are motiv să deschidă aceste fișiere, întrucât ele conțin credențiale fictive plasate intenționat ca momeală, astfel încât o singură tentativă de citire este suficientă pentru a confirma intenția ostilă. Indicatorii apar imediat în panoul de control, în același flux în care sunt afișate și celelalte evenimente, și confirmă post-factum că pivotarea a fost justificată. În plus, faptul că atacatorul a fost atras către aceste fișiere oferă telemetrie asupra tipurilor de credențiale pe care le urmărește, anume chei cloud, chei SSH și token-uri de CI/CD, informație utilă pentru evaluarea ulterioară a riscului și pentru anticiparea pașilor următori ai unei campanii reale.

6.5 Limitări

Această secțiune enunță explicit clasele de atac pe care sistemul nu le acoperă eficient. Scopul este de a delimita responsabilitatea KernelTrap și de a evita așteptări nerealiste asupra protecției oferite.

Atacatori avansați cu unelte proprii. Un actor de tip APT (*Advanced Persistent Threat*, atacator cu resurse și obiective de nivel statal sau corporativ) care folosește binare scrise sau modificate de el însuși poate ocoli o parte din indicatorii sistemului. Două categorii de tehnici sunt în mod special problematice. Prima: nume de proces alese deliberat pentru a semăna cu daemoni legitimi de sistem (de exemplu, un implant numit `kworker/0:1` sau `systemd-resolved-helper`) păcălește codificarea prin

frecvență a câmpului `processName` și lexicoanele de procese cunoscute. A doua: tehnici care nu lasă deloc urma caracteristică a unei comenzi noi, adică cod executat direct din memorie, fără ca un proces nou să fie creat prin syscall-ul `execve` (*fileless*); injectarea unei biblioteci dinamice ostile într-un proces deja existent și benign (*DLL/SO injection* în `sshd` sau `bash`); sau interpunerea pe syscall-urile unui proces de încredere prin `ptrace` (un mecanism de depanare folosit pentru a controla execuția altui proces). Aceste tehnici sunt caracteristice operatorilor cu resurse considerabile și nu constituie profilul de atacator țintă al sistemului.

Atacuri pre-autentificare. Agentul monitorizează exclusiv UID-urile cu sesiuni SSH interactive active (terminalele virtuale `pts/*`). În consecință, ies complet din raza sa de acoperire: brute-force-ul SSH (încercări automate de parole care nu produc o sesiune validă), exploit-urile care vizează direct procesul `sshd` înainte de stabilirea sesiunii, precum și atacurile asupra altor servicii expuse public (servere web, baze de date).

Exploit de kernel sau orbire a Tracee. Un atac care reușește să dezactiveze sau să modifice programele eBPF deja încărcate în kernel orbește complet agentul. Aceasta poate veni dintr-o vulnerabilitate *zero-day* în verificatorul eBPF (componenta de kernel care validează programele eBPF înainte de încărcare), din exploatarea capabilităților privilegiate `CAP_BPF` sau `CAP_SYS_ADMIN`, sau din înlocuirea pur și simplu a binarului Tracee. Toate aceste tehnici necesită oricum drepturi apropiate de `root`, situație în care protecția oferită se reduce la asumarea explicită din secțiunea 6.1 (contul `root` este considerat necompromis).

Vectori de atac în afara SSH. Execuția de cod la distanță printr-o aplicație web compromisă (*Remote Code Execution* pe Apache, Tomcat, servere de aplicații), sarcini malițioase rulate prin `cron`, sau exploit-uri ale unor servicii care rulează sub conturi de sistem alb-listate (`www-data` pentru servere web, `mysql`, `postgres` pentru baze de date) sunt invizibile filtrelor agentului, întrucât nu trec prin nicio sesiune SSH. KernelTrap este construit pentru scenariul „cont SSH neprivilegiat compromis”, nu pentru compromiterea generală a tuturor serviciilor expuse.

6.5.1 Domeniul de aplicabilitate optim

Pentru a clarifica unde KernelTrap oferă valoare maximă, tabelul 6.7 cartografiază principalele profile de atacator împotriva nivelului de acoperire al sistemului.

KernelTrap își propune să ridice semnificativ costul atacatorilor de nivel mediu segmentul în care se încadrează majoritatea atacurilor observate în trafic real, conform sondajelor de threat intelligence fără a pretinde acoperirea spectrului complet pe care un EDR enterprise îl tratează.

Tabela 6.7: Profile de atacator și acoperirea KernelTrap

Profil atacator	Acoperire	Motiv
Script kiddies / atacatori oportuniști	Optimă	Folosesc kit-uri publice (LinPEAS, LinEnum, pspy, module recon Metasploit) ușor recunoscute prin lexicoane.
Botneturi care compromit SSH prin credential stuffing	Optimă	După obținerea foothold-ului, execută în lanț scripturi automate de post-exploatare (recon, descărcare payload, persistență, lateral movement); generează rafale dense de syscall-uri și procese cu profil zgomotos, ușor de prins de fereastra glisantă.
Mișcare laterală prin cont SSH compromis	Optimă	Scenariul de proiectare; întreaga arhitectură este construită pentru acest caz.
APT cu tooling custom / fileless / 0-day kernel	Limitată	Indicatorii bazați pe nume de proces și syscall-uri standard nu rețin profilul.
Brute-force / exploit pre-autentificare	Niciuna	Filtrele pe UID SSH activ exclud această fază.
RCE prin aplicații web sau servicii expuse	Niciuna	UID-urile sistemice sunt alb-listate; vectorul nu trece prin SSH.

6.6 Sinteză evaluare

Evaluarea experimentală a confirmat că sistemul atinge ținta de overhead stabilită în capitolul 5 (sub 2,5% pe CPU pe gazda monitorizată) și detectează cu succes atât tooling-ul de recon automat (LinPEAS), cât și stabilirea unui canal C2 modern (Sliver), declanșând în fiecare caz pivotarea automată a sesiunii SSH compromise. Indicatorii binari de tip canary, observați după redirectionarea în honeypot, oferă confirmarea independentă a faptului că pivotarea a fost justificată și telemetrie suplimentară asupra obiectivelor atacatorului. Limitările identificate actori avansați cu tooling custom, atacuri pre-autentificare, vectori în afara SSH sunt asumate explicit și poziționează KernelTrap ca un sistem de detecție și răspuns activ specializat pe atacatori autentificați de nivel mediu, complementar soluțiilor de protecție perimetrare și EDR-urilor generale, nu un înlocuitor pentru acestea.

Capitolul 7

Concluzii și direcții viitoare

Lucrarea de față a tratat problema integrării a două abordări complementare de apărare cibernetică (detectia intruziunilor pe gazdă și honeypot-urile) într-un singur sistem care detectează automat un atacator autentificat prin SSH și îl redirectionează transparent într-un mediu izolat. Capitolul 2 a introdus fundamentele teoretice necesare: apelurile de sistem ca sursă de telemetrie, tehnologia eBPF cu instrumentul Tracee, algoritmul Isolation Forest, magistrala Redis Streams și containerele Docker. Capitolul 3 a analizat stadiul actual, acoperind sistemele HIDS bazate pe eBPF, metodele de învățare automată nesupervizată aplicate la detectia intruziunilor, honeypot-urile SSH cu interacțiune ridicată și mecanismele de apărare activă. Pe baza analizei, capitolul 4 a propus arhitectura sistemului KernelTrap și a justificat fiecare decizie de proiectare în raport cu literatura citată. Capitolul 5 a prezentat implementarea celor patru componente decuplate (agent, server central, container honeypot, panou de control) și modul lor de comunicare prin Redis Streams. Capitolul 6 a raportat evaluarea experimentală pe trei scenarii reale de atac, metricile de performanță măsurate și limitările asumate explicit.

Revenind la promisiunile enunțate în introducere, lucrarea confirmă livrarea celor trei niveluri funcționale anunțate. Colectarea evenimentelor de syscall prin eBPF cu Tracee a fost concretizată în agentul descris în subsecțiunea *Componente* a capitolului 5. Analiza centralizată cu un model Isolation Forest antrenat pe setul de date BETH a fost concretizată în clasa **Scorer** a serverului central, calibrată prin praguri obținute pe partiția de validare. Mecanismul de răspuns activ, adică combinația de semnal POSIX, regulă iptables și provizionare în container Docker, a fost concretizat în scripturile `hp_pivot_user.sh` și `hpTrap.sh`. Pivotarea transparentă, fără alertarea atacatorului, a fost demonstrată în cele trei scenarii reale din capitolul 6 (recon automat cu LinPEAS, stabilirea unui canal C2 cu Sliver, accesul la fișiere canary post-pivotare), iar ținta de overhead stabilită ca cerință nefuncțională (sub 2,5% pe CPU) a fost atinsă.

Dincolo de simpla însumare a componentelor, valoarea sistemului constă în bucla completă pe care o închide: detectia comportamentală furnizează semnalul, agregarea

temporală îl filtrează de zgomotul activității legitime, iar răspunsul activ transformă acest semnal într-o acțiune concretă de izolare. Fiecare verigă a fost necesară. Fără agregarea pe fereastră glisantă, scorurile izolate ar fi produs pivotări eronate; fără pivotarea transparentă, detecția ar fi rămas o simplă alertă, lăsând atacatorul pe sistemul real; fără honeypot, sesiunea redirectionată nu ar fi avut unde să fie observată în continuare. Tocmai această dependență reciprocă a componentelor justifică tratarea lor ca un sistem unitar, și nu ca module independente.

Limitările sistemului (atacatori APT cu unelte proprii, atacuri pre-autentificare, exploit-uri de kernel și vectori în afara SSH) au fost asumate explicit în secțiunea omonimă a capitolului 6 și nu reprezintă o omisiune, ci o decizie deliberată privind domeniul de aplicabilitate. KernelTrap nu pretinde acoperirea spectrului complet de atacuri, ci abordarea eficientă a unei clase bine definite: atacatorul autentificat de nivel mediu pe o gazdă altfel sănătoasă.

Pe partea de direcții viitoare, sistemul construit reprezintă o bază solidă pe care se pot construi mai multe extinderi naturale, dintre care cele mai promițătoare sunt următoarele:

1. Detecție hibridă: semnături combinate cu comportament. Combinarea modelului Isolation Forest cu un strat de reguli declarative (Sigma, YARA) ar permite acoperirea simultană a două categorii de atac: cele cunoscute, capturate cu precizie ridicată de semnături, și cele necunoscute sau anormale, capturate de model. Stratul de semnături poate funcționa ca o cale rapidă care scurtcircuitează scoringul ML atunci când un tipar cunoscut este recunoscut, reducând atât latența cât și rata de fals pozitive.
2. Honeypot configurabil per scenariu. Containerul honeypot este în prezent o imagine Ubuntu generică, cu un set fix de fișiere-momeală. O extindere naturală constă în introducerea de profile selectabile (server web cu fișiere de configurare Nginx fictive, server de baze de date cu dumpuri SQL momeală, workstation de dezvoltare cu repository-uri Git ce conțin credențiale inserate intenționat), alese automat în funcție de profilul gazdei monitorizate sau de comportamentul observat al atacatorului.
3. Extindere către o suită HIPS / NIPS / EDR. KernelTrap acoperă în prezent doar stratul HIPS, axat pe gazdă. O direcție de dezvoltare semnificativă ar fi adăugarea unui modul NIPS (*Network Intrusion Prevention System*) care să analizeze fluxul de pachete dintre agenți, respectiv a unor capabilități tipice EDR (*Endpoint Detection and Response*) precum monitorizarea integrității fișierelor critice de sistem și jurnalizarea per-proces a accesului la fișiere sensibile.
4. Corelare cross-host pentru detecția mișcării laterale. Arhitectura curentă tratează

fiecare gazdă independent; un atacator care pivotează succesiv prin mai multe servere generează semnale separate, fără o vedere unificată. Adăugarea unui modul de corelare pe serverul central, care să grupeze evenimente după identitate (cheie SSH reutilizată, IP sursă, proximitate temporală), ar permite detecția pattern-urilor de tip multi-step intrusion și emiterea unei comenzi de pivotare simultan pe toate gazdele afectate.

5. Integrare cu surse externe de threat intelligence. Conectarea sistemului la feed-uri publice de tip MISP sau AlienVault OTX ar permite popularea automată a blocklist-urilor `iptables` cu IP-uri ostile cunoscute și îmbogățirea panoului de control cu context suplimentar (categoria atacatorului, campanie atribuită, indicatori asociați). În plus, datele colectate în propriul honeypot ar putea fi publicate înapoi în feed-uri, contribuind la ecosistemul de partajare.
6. Honeypot augmentat cu modele lingvistice (arie de cercetare). O direcție exploratorie este utilizarea modelelor lingvistice mari (LLM) pentru generarea dinamică a răspunsurilor în honeypot (ieșiri credibile la `cat`, `ls` sau `ps`), făcând mediul-momeală mai convingător pentru atacatorii umani. Lucrări recente, precum cea a lui Sladić et al. [44], explorează această direcție și pot oferi un punct de plecare pentru o eventuală integrare.

În sinteză, lucrarea de față a contribuit cu un sistem distribuit funcțional care demonstrează fezabilitatea integrării detecției comportamentale cu un răspuns activ izolant, oferind atât o platformă utilizabilă în practică pe gazde Linux protejate, cât și un punct de plecare concret pentru extinderile enumerate mai sus. KernelTrap se poziționează astfel ca o componentă complementară soluțiilor perimetrare și EDR-urilor enterprise, nu ca un înlocuitor pentru acestea.

Bibliografie

- [1] A comprehensive survey on cyber deception techniques to improve honeypot performance. *Computers & Security*, 140:103792, 2024.
- [2] Honeypots and honeytokens in active defense. *ResearchGate*, 2024. Publication 377927971.
- [3] Real-time intrusion detection and prevention with neural network in kernel using eBPF. In *Proceedings of the IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, 2024.
- [4] Advancing cybersecurity with honeypots and deception strategies. *Informatics*, 12(1):14, 2025.
- [5] AI-powered intrusion detection system with honeypot integration. *ResearchGate*, 2025. Publication 395500692.
- [6] eBPF-PATROL: Protective agent for threat recognition and overreach limitation. *arXiv preprint arXiv:2511.18155*, 2025.
- [7] One-class anomaly detection for industrial applications: A comparative survey and experimental study. *Computers*, 14(7):281, 2025.
- [8] A survey of streaming data anomaly detection in network security. *PeerJ Computer Science*, 2025.
- [9] Q-Cowrie: An adaptive honeypot to analyse attackers' behaviour. *International Journal of Information Security*, 2026.
- [10] K. Anand et al. Robust IoT security using isolation forest and one class SVM algorithms. *Scientific Reports*, 2025.
- [11] Aqua Security. Tracee events reference. <https://aquasecurity.github.io/tracee/latest/docs/events/>, 2024. Accesat: 2026.
- [12] Aqua Security. Tracee: Linux runtime security and forensics using eBPF. <https://github.com/aquasecurity/tracee>, 2024. Accesat: 2026.

- [13] Samer Atawneh, Aljawharah Aljehani, and Reem Aljehani. Harnessing AI for cyber defense: Honeypot-driven intrusion detection systems. *Symmetry*, 17(5):628, 2025.
- [14] Daniel Bourke and Daniel Grzelak. Breach detection at scale with AWS honey tokens. In *Black Hat Asia*, 2018.
- [15] Daniel P. Bovet and Marco Cesati. *Understanding the Linux Kernel*. O'Reilly Media, 3 edition, 2005.
- [16] Cao et al. Anomaly detection and enterprise security using user and entity behavior analytics (UEBA). In *IEEE International Conference Publication*, 2023.
- [17] Varun Chandola, Arindam Banerjee, and Vipin Kumar. Anomaly detection: A survey. *ACM Computing Surveys*, 41(3):15:1–15:58, 2009.
- [18] J. Cui, G. Zhang, Z. Chen, and N. Yu. Multi-homed abnormal behavior detection algorithm based on fuzzy particle swarm cluster in UEBA. *Scientific Reports*, 2022.
- [19] Mohamed-Cherif Dani, François-Xavier Jollois, Mohamed Nadif, and Cassio Freixo. Adaptive threshold for anomaly detection using time series segmentation. In *Lecture Notes in Computer Science (AINA)*. Springer, 2016.
- [20] Z. Ding and M. Fei. An anomaly detection approach based on isolation forest algorithm for streaming data using sliding window (iForestASD). *arXiv preprint*, 2024.
- [21] Evgeny Eremin. Unsupervised anomaly detection on cybersecurity data streams: a case with BETH dataset. *arXiv preprint arXiv:2503.04178*, 2025.
- [22] Guillaume Fournier, Sylvain Afchain, and Sylvain Baubeau. Runtime security monitoring with eBPF. In *Proceedings of SSTIC 2021*, 2021.
- [23] Brendan Gregg. *BPF Performance Tools: Linux System and Application Observability*. Addison-Wesley Professional, 2019.
- [24] Jinghao Hao, Pradeep Subramaniam, Sathya Kannan, et al. The eBPF runtime in the Linux kernel. *arXiv preprint arXiv:2410.00026*, 2024.
- [25] Kate Highnam, Kai Arulkumaran, Zachary Hanif, and Nicholas R. Jennings. BETH dataset: Real cybersecurity data for anomaly detection research. In *ICML Workshop on Uncertainty and Robustness in Deep Learning*, 2021.

- [26] Kate Highnam, Zachary Hanif, Eric Van Vogt, Sonali Parbhoo, Sergio Maffei, and Nicholas R. Jennings. Adaptive experimental design for intrusion data collection. *arXiv preprint arXiv:2310.13224*, 2023.
- [27] Celestine Iwendi et al. Containerized cloud-based honeypot deception for tracking attackers. *Scientific Reports*, 2023.
- [28] Marco Kahlhofer and Stefan Rass. Application layer cyber deception without developer interaction. In *IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*, pages 416–429, 2024.
- [29] Michael Kerrisk. *The Linux Programming Interface*. No Starch Press, 2010.
- [30] Lalon Phurba Khan, Akib Hossain, and Subir Dey. Cyber attack detection using deep multi-agent reinforcement learning with BETH dataset. *SN Computer Science*, 2025.
- [31] Bhanu Lakha, Sarah L. Mount, Edoardo Serra, and Alfredo Cuzzocrea. Anomaly detection in cybersecurity events through graph neural network and transformer based model: A case study with BETH dataset. In *Proceedings of the 2022 IEEE International Conference on Big Data*, 2022.
- [32] Sang Yong Lim et al. LIGHT-HIDS: A lightweight and effective machine learning-based framework for robust host intrusion detection. *arXiv preprint arXiv:2509.13464*, 2025.
- [33] Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. Isolation forest. In *Eighth IEEE International Conference on Data Mining (ICDM)*, pages 413–422. IEEE, 2008.
- [34] Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. Isolation-based anomaly detection. *ACM Transactions on Knowledge Discovery from Data (TKDD)*, 6(1):1–39, 2012.
- [35] Ming Liu, Zhi Xue, Xianghua Xu, Changmin Zhong, and Jinjun Chen. Host-based intrusion detection system with system calls: Review and future trends. *ACM Computing Surveys*, 51(5):98:1–98:36, 2018.
- [36] Pedro Beltrán Lopez, Pantaleone Nespole, and Manuel Gil Pérez. Cyber deception reactive: TCP stealth redirection to on-demand honeypots. *arXiv preprint arXiv:2402.09191*, 2024.
- [37] Steven McCanne and Van Jacobson. The BSD packet filter: A new architecture for user-level packet capture, 1993.

- [38] Dirk Merkel. Docker: Lightweight linux containers for consistent development and deployment. *Linux Journal*, 2014(239), 2014.
- [39] Paulin K. Mvula, Paula Branco, Guy-Vincent Jourdan, and Herna L. Viktor. Evaluating word embedding feature extraction techniques for host-based intrusion detection systems. *Discover Data*, 2023.
- [40] Redis Ltd. Redis streams. <https://redis.io/docs/latest/develop/data-types/streams/>, 2024. Accesat: 2026.
- [41] Reworr and Dmitrii Volkov. LLM agent honeypot: Monitoring AI hacking agents in the wild. *arXiv preprint arXiv:2410.13919*, 2024.
- [42] John H. Ring, Colin M. Van Oort, Samson Durst, Vanessa White, Joseph P. Near, and Christian Skalka. Methods for host-based intrusion detection with deep learning. *Digital Threats: Research and Practice*, 2(4):26:1–26:29, 2021.
- [43] A. Satpute, S. Nikam, V. Gaikwad, Y. Kakade, and C. Mhaske. AI-driven intrusion detection system using SSH honeypots. *ICCK Transactions on Cybersecurity*, 1(1):3–12, 2025.
- [44] Muris Sladić, Veronica Valeros, Carlos Catania, and Sebastian Garcia. LLM in the shell: Generative honeypots. In *Proceedings of the IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*, pages 430–435, 2024.
- [45] David Soldani, Petrit Nahi, Hadi Bour, Saeid Jafarizadeh, Mohammed F. Soliman, Luca Di Giovanna, Federico Monaco, Giuseppe Ognibene, and Fulvio Riso. eBPF: A new approach to cloud-native observability, networking and security for current (5G) and future mobile networks (6G and beyond). *IEEE Access*, 11:57174–57202, 2023.
- [46] Lance Spitzner. *Honeypots: Tracking Hackers*. Addison-Wesley, 2002.
- [47] A. A. Syairozi and Arizal. Comparative analysis of eBPF-based runtime security monitoring tools in monitoring and threat detection on Kubernetes. In *Proceedings of RITECH 2025*. SCITEPRESS, 2025.
- [48] J. Užupis, A. Brilingaitė, et al. Risk-based system-call sequence grouping method for malware intrusion detection. *Electronics*, 13(1):206, 2024.
- [49] Tao Yu, Yang Xin, and Chen Zhang. HoneyFactory: Container-based comprehensive cyber deception honeynet architecture. *Electronics*, 13(2):361, 2024.

- [50] Mohammed Yunus, Aizat Mohd Ali, Rasidah Osman, and José Antonio Iglesias Martínez. Online and unsupervised anomaly detection for streaming data using an array of sliding windows. *arXiv preprint*, 2024.
- [51] Tommaso Zoppi and Andrea Ceccarelli. Prepare for trouble and make it double! Supervised–Unsupervised stacking for anomaly-based intrusion detection. *Journal of Network and Computer Applications*, 2022.
- [52] Tommaso Zoppi, Andrea Ceccarelli, et al. Unsupervised anomaly detectors to detect intrusions in the current threat landscape. *ACM Transactions on Dependable and Secure Computing*, 2022.